



Contents lists available at ScienceDirect

Journal of the Franklin Institute

journal homepage: www.elsevier.com/locate/fi

Reinforcement learning based attitude fault-tolerant control of spacecraft with unknown system model

Shaolong Yang^a, Lei Jin^{b,*}, Jiaxuan Rao^b

^a Shanghai Institute of Spaceflight Control Technology, Shanghai 201108, China

^b Beihang University, School of Astronautics, Beijing 102206, China

ARTICLE INFO

Keywords:

Spacecraft
Reinforcement learning
Fault-tolerant control
Actuator fault

ABSTRACT

With the increasing reliability and safety requirements of spacecraft control system, it is urgent to study effective fault-tolerant control methods to ensure that the control system can still maintain high control performance when the actuator fails. Considering the uncertainty and suddenness of faults, it is very important for fault-tolerant control to have strong adaptability and high real-time performance. Therefore, a fault-tolerant attitude controller based on reinforcement learning for a spacecraft with unknown system model is proposed in this paper. Firstly, by ignoring the effects of inertia uncertainty, actuator faults, and external disturbances, the reinforcement learning algorithm is combined with optimal control to design an offline approximate optimal control policy for the known nominal system. Next, a neural network-based observer is designed to estimate the system input matrix and approximate the unknown system dynamics, eliminating the offline nominal controller's dependency on the system model and mitigating the impact of multiplicative actuator faults on attitude control. Subsequently, the offline nominal controller is employed as the initial control strategy, and the parameters of the critic network are updated online using the RLS method. This approach results in an online reinforcement learning controller that enhances the algorithm's real-time performance. In addition, by utilizing the partial disturbances estimated by the neural network-based observer, a feedforward compensation algorithm is incorporated to counteract the adverse effects of additive actuator faults and external disturbance torques on attitude control performance, completing the online fault-tolerant control scheme based on reinforcement learning. Lastly, the stability of the control system is proved by the Lyapunov method, and the validity of the proposed fault-tolerant control is illustrated through simulations.

1. Introduction

Since the inception of human space exploration, numerous spacecraft failures have been documented in orbit. Notable examples include the American navigation satellite GPS BII-7, the Japanese X-ray astronomy satellite Astro-H, and the American commercial remote sensing satellite WorldView-4 [1]. The harsh and unique nature of the space environment poses significant challenges for in-orbit repairs, which are not as readily achievable as repairs to ground-based systems. To enhance the safety and reliability of spacecraft, fault-tolerant control strategies have been developed and implemented.

The thoughts of fault-tolerant control originated from the integrity control first proposed by Niederlinski in 1971 [2], and the

* Corresponding author.

E-mail address: jinlei@buaa.edu.cn (L. Jin).

concept of fault-tolerant control was officially proposed by the American IEEE Control Systems Society in 1986 [3]. A control system with the ability to automatically adapt to component faults is called a fault-tolerant control system. According to the different ways of handling faults, fault-tolerant control can be divided into two categories: passive fault-tolerant control and active fault-tolerant control [4]. Passive fault-tolerant control uses the robustness of the controller to keep the system stable when preset faults occur. Jin et al. [5] designed a passive fault-tolerant controller using dynamic inversion and time-delay control theory, thereby achieving spacecraft attitude control in the presence of actuator faults. Zhang et al. [6] introduced the historical control input into the control system and proposed a fault-tolerant control scheme for online learning, which improved the robustness of the controller. Passive fault-tolerant control systems are characterized by their simple structure and high reliability. However, they have poor adaptability and may experience a significant degradation in control performance when confronted with unknown and complex faults. Unlike passive fault-tolerant control, active fault-tolerant control dynamically adapts the control strategy in real-time based on detected faults to maintain system stability and performance. Zhou et al. [7] designed two fault observers to estimate the magnitudes of additive and multiplicative faults, respectively, and achieved stability control of spacecraft attitude through a control law and compensation algorithm. Shen et al. [8] utilized both the estimated and measured values of angular velocities for fault detection and estimation, and combined these with an integral backstepping control method to achieve stable control of spacecraft attitude. Active fault-tolerant control provides high adaptability and performance by dynamically adjusting to detected faults, but it requires complex monitoring and computational resources, increasing implementation costs and system complexity.

Compared to passive and active fault-tolerant control methods, adaptive control, which does not require precise fault information and offers strong adaptability and robustness, has also been widely applied in the field of fault-tolerant control. Wang et al. [9] designed an adaptive integral terminal sliding mode fault-tolerant controller to enhance the stability of the spacecraft attitude tracking control process. Tutsoy et al. [10] combined an adaptive robust reduced-order observer with nonlinear dynamic inversion control to achieve fault-tolerant control of quad-rotor UAVs. Adaptive technology enhances the adaptability of fault-tolerant control but still relies heavily on system models. To address this dependency, neural networks are employed to estimate uncertain models. Sanwale et al. [11] and Zhou et al. [12] used neural networks to estimate the magnitude of faults and applied the adaptive sliding mode control method to achieve stability control of spacecraft attitude, which improved system reliability and control performance. However, neural networks require substantial computational resources and complex training processes, and they depend on large volumes of high-quality data.

In recent years, advancements in computer hardware performance and the development of efficient algorithms have significantly propelled the growth of reinforcement learning. Reinforcement learning enables agents to learn policies that maximize long-term rewards through continuous interaction with their environment [13]. Building on these advancements, reinforcement learning-based fault-tolerant control has emerged as a promising approach, facilitating model-free control through ongoing environmental interaction. Li et al. [14] employed a model-free DDPG reinforcement learning algorithm to train a fault-tolerant control method for manipulators, resulting in a controller that is highly insensitive to joint faults. Ahmed and Khorasgani [15] proposed a hierarchical reinforcement learning-based fault-tolerant controller which was verified using a C-130 aircraft fuel tank model. To enhance the adaptability of the controller, Ahmed and Quiñones-Grueiro [16] combined online and offline policies to propose an adaptive reinforcement learning algorithm that allows the controller to be updated in real-time according to system changes. Zhang and Gao [17] draw on the idea of model reference control and uses reinforcement learning algorithms to track state responses of non-fault systems regardless of whether faults occur or not. Ren et al. [18] proposed an incremental reinforcement learning strategy for flight control systems, gradually approximating optimal fault compensation. Zheng et al. [19] combined reinforcement learning with sliding mode observers for spacecraft attitude hyperagile tracking control, maintaining high accuracy under uncertainties. Reinforcement learning offers high adaptability and self-learning capabilities, making it suitable for complex and uncertain environments. However, it faces challenges including low sample efficiency, slow convergence, and difficulty in proving theoretical stability.

To address the challenge of proving theoretical stability in reinforcement learning, current research combines traditional control theories with reinforcement learning. This integration leverages the stability-guaranteed framework of traditional control and the adaptability of reinforcement learning to unknown faults and disturbances. The resulting hybrid architecture is particularly effective in complex scenarios, ensuring theoretical reliability while enhancing autonomous decision-making in uncertain environments. Hua et al. [20] proposed a data-driven reinforcement learning control algorithm that achieves fault-tolerant control of DC motors. Li et al. [21] proposed a data-driven reinforcement learning method to address model-free fault-tolerant control for discrete-time multi-mode systems with actuator faults, achieving stability control while optimizing energy consumption. Wang et al. [22] proposed a reinforcement learning-based optimal fault-tolerant control method that achieves stable control in the presence of actuator faults and external disturbances by leveraging system data, thereby reducing the controller's reliance on precise models. Li et al. [23] proposed an innovative control method for industrial processes that combines offline reinforcement learning and minimax optimization, offering a model-free, highly fault-tolerant, and robust solution. For tracking control problems of multi-agent systems with actuator faults, Li et al. [24] combined reinforcement learning with backstepping control to design an adaptive fault-tolerant controller with compensation for the impact of faults through auxiliary signals. Zhao et al. [25] combined reinforcement learning with optimal control to design a fault-tolerant controller for unknown quadrotors, which improved control performance. Zhang et al. [26] combined reinforcement learning with fuzzy control to improve the robustness of the controller and successfully applies it to pitch rate control of F-16 fighters and robot arm systems. Liu et al. [27] proposed a fault-tolerant control method based on reinforcement learning, which achieves stable control of quadrotor UAVs under actuator faults.

In summary, current spacecraft attitude fault-tolerant control techniques still face several limitations, including insufficient intelligence in autonomous decision-making, limited adaptability to handle unknown faults and disturbances, and suboptimal real-time performance during abrupt actuator faults. These shortcomings highlight the need for more intelligent, adaptive, and computationally

efficient real-time fault-tolerant control strategies to ensure reliable spacecraft operation in unpredictable space environments.

Therefore, this paper combines reinforcement learning with optimal control method to design an online reinforcement learning-based fault-tolerant controller for spacecraft with unknown system model. Firstly, by ignoring the influence of inertia uncertainty, actuator faults and disturbance torques, the nominal systems model can be assumed to be known and an offline nominal controller based on reinforcement learning is proposed. This controller is an approximation of a nonlinear optimal controller that improves the intelligence and autonomy of the controller. Then, a neural network-based observer is designed to estimate the real system model parameters, in which the estimated input matrix containing the multiplicative actuator faults can be used in the offline approximate optimal control policy. Furthermore, the online reinforcement learning-based controller is developed by using the offline nominal controller as the initial control strategy and updating the parameters of the critic network online via the Recursive Least Squares (RLS) algorithm. RLS can update parameters online with good real-time performance and is widely used in system identification, signal processing, adaptive control, fault diagnosis, and fault-tolerant control, though its application in reinforcement learning is rare. The online reinforcement learning controller, based on RLS parameter updates, effectively reduces the impact of multiplicative actuator faults and achieves good real-time performance. Lastly, the observer-based feedforward compensation algorithm is used to eliminate the adverse effects of actuator additive faults and external disturbances. Based on the above design steps, the online fault-tolerant control based on reinforcement learning is achieved. In this fault-tolerant control scheme, the reinforcement learning algorithm replaces traditional model-based control algorithms, achieving spacecraft attitude stabilization control without requiring detailed system model information and while maintaining good real-time performance.

The main contributions compared to other existing works are stated as follows:

- 1) By integrating reinforcement learning with optimal control, the stability of the system is guaranteed, while the intelligence, autonomy and adaptability of the controller are improved.
- 2) The multiplicative faults of the actuator are treated as unknown model parameters and estimated using a neural network-based observer. Subsequently, the impact of these multiplicative faults on the spacecraft attitude is indirectly mitigated through the reinforcement learning algorithm.
- 3) The RLS algorithm is utilized to update the parameters of the critic network in reinforcement learning, offering both implementation convenience and enhanced real-time performance. Additionally, the RLS algorithm is initialized with the results from the offline controller, eliminating the need for an initial data collection phase.

The remainder of the paper is scheduled as follows: [Section 2](#) gives a problem statement where a system model is established including spacecraft attitude kinematics based on quaternions, spacecraft attitude dynamics and actuator faults models. [Section 3](#) combines reinforcement learning with optimal control theory to design an online fault-tolerant controller. [Section 4](#) presents the simulations and comparisons. [Section 5](#) concludes this research.

2. Problem formulation and preliminaries

2.1. Attitude kinematic and dynamic model of spacecraft

To avoid computational singularity, the spacecraft attitude is described using quaternions. Therefore, the attitude kinematic equations of the spacecraft can be expressed as

$$\begin{cases} \dot{q}_{b0} = -\frac{1}{2}\mathbf{q}_b^T \boldsymbol{\omega}_b \\ \dot{\mathbf{q}}_b = \frac{1}{2}(\tilde{\mathbf{q}}_b + q_{b0}\mathbf{I}_3)\boldsymbol{\omega}_b \end{cases} \quad (1)$$

where, $\mathbf{Q}_b = [q_{b0} \mathbf{q}_b^T]^T \in \mathbb{R} \times \mathbb{R}^3$ denotes the attitude orientation of the spacecraft in the body coordinate system relative to the inertial coordinate system, $q_{b0} \in \mathbb{R}$ is the scalar component and $\mathbf{q}_b = [q_1 \ q_2 \ q_3]^T \in \mathbb{R}^3$ is the vector component. $\mathbf{I}_3$ denotes third-order identity matrix. $\boldsymbol{\omega}_b = [\omega_x \ \omega_y \ \omega_z]^T \in \mathbb{R}^3$ denotes the angular velocity of the spacecraft in the body coordinate system relative to the inertial coordinate system. Under the action of the attitude kinematic equations, the attitude $\mathbf{Q}_b$ is obtained by inputting the attitude angular velocity $\boldsymbol{\omega}_b$. For any 3-dimensional vector $\mathbf{x} = [x_1 \ x_2 \ x_3]^T$, $\tilde{\mathbf{x}}$ is a cross product matrix, with

$$\tilde{\mathbf{x}} = \begin{bmatrix} 0 & -x_3 & x_2 \\ x_3 & 0 & -x_1 \\ -x_2 & x_1 & 0 \end{bmatrix}$$

Considering the space environment disturbance, the attitude dynamic equation of a rigid spacecraft is established as

$$\mathbf{I}_b \dot{\boldsymbol{\omega}}_b + \tilde{\boldsymbol{\omega}}_b \mathbf{I}_b \boldsymbol{\omega}_b = \mathbf{T}_c + \mathbf{T}_d \quad (2)$$

where, $\mathbf{I}_b \in \mathbb{R}^{3 \times 3}$ denotes the actual rotational inertial matrix of the spacecraft, which is usually a positive definite symmetric matrix. $\mathbf{T}_c \in \mathbb{R}^3$ refers to the control torque acting on the spacecraft, $\mathbf{T}_d \in \mathbb{R}^3$ refers to the external disturbance torques.

Spacecraft will consume fuel during missions such as attitude and orbit maneuvers, which will cause changes in the rotational inertia of the spacecraft and result in the actual and nominal rotational inertia of the spacecraft not being equal. Taking into account the impact of uncertainty in rotational inertia, the attitude dynamic equation of a rigid spacecraft can be given by

$$(\mathbf{I}_{b0} + \Delta\mathbf{I}_b)\dot{\boldsymbol{\omega}}_b + \tilde{\boldsymbol{\omega}}_b(\mathbf{I}_{b0} + \Delta\mathbf{I}_b)\boldsymbol{\omega}_b = \mathbf{T}_c + \mathbf{T}_d \quad (3)$$

where, $\mathbf{I}_{b0} \in \mathbb{R}^{3 \times 3}$ denotes the nominal rotational inertial matrix of the spacecraft, which is usually a positive definite symmetric matrix. $\Delta\mathbf{I}_b \in \mathbb{R}^{3 \times 3}$ refers to unknown additional rotational inertia.

Rearranging Eq. (3), the attitude dynamic equation of a rigid spacecraft can be rewritten as

$$\mathbf{I}_{b0}\dot{\boldsymbol{\omega}}_b + \tilde{\boldsymbol{\omega}}_b\mathbf{I}_{b0}\boldsymbol{\omega}_b = \mathbf{T}_c + \mathbf{T}_d + \mathbf{T}_\Delta \quad (4)$$

where, $\mathbf{T}_\Delta = -\Delta\mathbf{I}_b\dot{\boldsymbol{\omega}}_b - \tilde{\boldsymbol{\omega}}_b\Delta\mathbf{I}_b\boldsymbol{\omega}_b$ is unknown torque generated by uncertainty in inertia.

2.2. Attitude dynamic model of spacecraft considering actuator faults

The control torque acting on the spacecraft is generated by the actuator. Although there are many types of actuators, the mathematical model of actuators can be uniformly expressed as

$$\mathbf{T}_c = \mathbf{C}\mathbf{u} \quad (5)$$

where, $\mathbf{C} \in \mathbb{R}^{3 \times n}$ is the actuator effectiveness matrix, $\mathbf{u} = [u_1 \ u_2 \ \dots \ u_n]^T$ refers to the control torque vector generated by the actuators, n refers to the number of the actuators.

Considering the impact of actuator faults, the relationship between the actual control torque generated by the actuator and the expected control torque can be expressed as

$$\mathbf{u} = (\mathbf{I}_n - \mathbf{E})\mathbf{u}_c + \bar{\mathbf{u}}_c \quad (6)$$

where, $\mathbf{u}_c = [u_{c1} \ u_{c2} \ \dots \ u_{cn}]^T$ refers to the expected control torque, $\bar{\mathbf{u}}_c = [\bar{u}_{c1} \ \bar{u}_{c2} \ \dots \ \bar{u}_{cn}]^T$ refers to the additive faults, $\mathbf{E} = \text{diag}(e_1 \ e_2 \ \dots \ e_n)$ is a diagonal matrix composed of actuator fault coefficients, e_i represents the fault coefficient of the i th actuator, with $e_i \in [0, 1]$. $e_i = 0$, means that the actuator is working well; and $e_i = 1$, means the actuator totally loses its effectiveness; and $0 < e_i < 1$, implies that the actuator partially loses its effectiveness.

The presence of both additive and multiplicative faults in the actuators causes the actual control torque to deviate from the expected control torque, thereby affecting the system's dynamic response. Moreover, these actuator faults introduce additional model uncertainties, which not only reduce the system's robustness but also significantly degrade its control performance. In severe cases, this deviation can lead to system instability.

Substituting Eq. (5) and Eq. (6) into Eq. (4), the attitude dynamic equation considering environmental disturbance, inertia uncertainty and actuator faults can be expressed as:

$$\mathbf{I}_{b0}\dot{\boldsymbol{\omega}}_b + \tilde{\boldsymbol{\omega}}_b\mathbf{I}_{b0}\boldsymbol{\omega}_b = \mathbf{T}_{cf} + \mathbf{T}_f + \mathbf{T}_d + \mathbf{T}_\Delta \quad (7)$$

where, $\mathbf{T}_{cf} = \mathbf{C}\mathbf{u}_c$ refers to the expected control torque generated by the actuators, $\mathbf{T}_f = -\mathbf{C}\mathbf{E}\mathbf{u}_c + \mathbf{C}\bar{\mathbf{u}}_c$ refers to the additional control torque generated by the actuator faults.

2.3. System state equation

By defining system state variables as $\mathbf{x} = [\mathbf{q}_b^T \ \boldsymbol{\omega}_b^T]^T$, based on Eqs.(1), (2), (5) and (6), system state equation is described by

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{B}\mathbf{u}_c + \boldsymbol{\delta} \quad (8)$$

where, $\mathbf{f}(\mathbf{x})$ denotes the nonlinear dynamics of the system and satisfies the Lipschitz condition, $\mathbf{B}$ refers to the input matrix, $\boldsymbol{\delta}$ refers to the partial disturbances, which contain additive faults, disturbance torques and a part of rotational inertia effects, with

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} \mathbf{G}(\mathbf{q}_b)\boldsymbol{\omega}_b \\ -\mathbf{I}_b^{-1}\tilde{\boldsymbol{\omega}}_b\mathbf{I}_b\boldsymbol{\omega}_b \end{bmatrix}, \mathbf{B} = \begin{bmatrix} \mathbf{0}_{3 \times n} \\ \mathbf{I}_b^{-1}\mathbf{C}(\mathbf{I}_n - \mathbf{E}) \end{bmatrix}, \boldsymbol{\delta} = \begin{bmatrix} \mathbf{0}_{3 \times 1} \\ \mathbf{I}_b^{-1}(\mathbf{C}\bar{\mathbf{u}}_c + \mathbf{T}_d) \end{bmatrix} \quad (9)$$

where, $\mathbf{G}(\mathbf{q}_b) = \frac{1}{2}(\tilde{\mathbf{q}}_b + (1 - \mathbf{q}_b^T\mathbf{q}_b)\mathbf{I}_3)$

If all the variables in Eq. (8) are known, the system model is known. The controller can be designed by traditional methods, such as sliding mode control, backstepping control and optimal control. However, when the spacecraft has model uncertainty and actuator faults, the terms $\mathbf{I}_b$, $\mathbf{E}$, $\bar{\mathbf{u}}_c$, and $\mathbf{T}_d$ are all unknown, which also leads to uncertainty in terms $\mathbf{f}(\mathbf{x})$, $\mathbf{B}$, and $\boldsymbol{\delta}$ in the system equation. Specifically, the spacecraft's rotational inertia uncertainty affects $\mathbf{f}(\mathbf{x})$, $\mathbf{B}$, and $\boldsymbol{\delta}$, multiplicative actuator faults affects $\mathbf{B}$, and additive actuator faults and disturbance torques will affect $\boldsymbol{\delta}$.

Therefore, the goal is to design an attitude stabilization controller that is bounded stable even if $\mathbf{f}(\mathbf{x})$, $\mathbf{B}$ and $\boldsymbol{\delta}$ of the system model

described by Eq. (8) are unknown.

3. Control law design

3.1. Offline nominal controller based on reinforcement learning

3.1.1. Optimal controller for nominal nonlinear systems

Ignoring model uncertainty, actuator faults and external disturbance torques, the state equation of the nominal system can be expressed as

$$\dot{\mathbf{x}} = \mathbf{f}_0(\mathbf{x}) + \mathbf{B}_0 \mathbf{u}_c \quad (10)$$

where, $\mathbf{f}_0(\mathbf{x})$ denotes the internal nonlinear dynamics of the system ignoring model uncertainty, $\mathbf{B}_0$ refers to the input matrix, with

$$\mathbf{f}_0(\mathbf{x}) = \begin{bmatrix} \mathbf{G}(\mathbf{q}_b)\boldsymbol{\omega}_b \\ -\mathbf{I}_{b0}^{-1}\boldsymbol{\omega}_b\mathbf{I}_{b0}\boldsymbol{\omega}_b \end{bmatrix}, \mathbf{B}_0 = \begin{bmatrix} \mathbf{0}_{3 \times n} \\ \mathbf{I}_{b0}^{-1}\mathbf{C} \end{bmatrix} \quad (11)$$

Consider control performance and control consumption, define the reward function as follows

$$r_1(\mathbf{x}, \mathbf{u}_c) = \mathbf{x}^T \mathbf{Q}_1 \mathbf{x} + \mathbf{u}_c^T \mathbf{R}_1 \mathbf{u}_c \quad (12)$$

where, $\mathbf{Q}_1$ and $\mathbf{R}_1$ are positive-definite matrices with appropriate dimensions, representing the weights of control performance and control input.

Based on Eq. (12), define the cost function as follows

$$J_1(t) = \int_t^\infty r_1(\tau) d\tau \quad (13)$$

Therefore, the optimal control problem can be stated as how to determine the control policy to minimize the value of the cost function $J_1(t)$.

For the nonlinear nominal system described in Eq. (10), a continuous control policy is considered admissible if it results in zero control output when the state is zero and ensures system stability while keeping the cost function as Eq. (13) bounded.

According to optimal control theory, for any admissible control policy, if the cost function as Eq. (13) is continuously differentiable, then the Hamilton function of the above optimal control problem can be defined as

$$\begin{aligned} H(\mathbf{x}, \mathbf{u}_c, \nabla J_1) &= r_1 + (\nabla J_1)^T \dot{\mathbf{x}} \\ &= r_1 + (\nabla J_1)^T (\mathbf{f}_0(\mathbf{x}) + \mathbf{B}_0 \mathbf{u}_c) \end{aligned} \quad (14)$$

where, $\nabla J_1 = \partial J_1 / \partial \mathbf{x}$ refers to the partial derivative of the cost function with respect to the state variables.

According to the Bellman optimality principle, the optimal cost function J_1^* satisfies the Hamilton-Jacobi-Bellman (HJB) equation

$$\min_{\mathbf{u}_c} H(\mathbf{x}, \mathbf{u}_c, \nabla J_1^*) = 0 \quad (15)$$

Substituting Eq. (12) into Eq. (14) and solving for the partial derivative of H with respect to $\mathbf{u}_c$, $\partial H / \partial \mathbf{u}_c = 2\mathbf{R}_1 \mathbf{u}_c + \mathbf{B}_0^T \nabla J_1$ can be obtained. Assuming that the minimum value on the left side of the above HJB equation exists and is unique, then $\partial H / \partial \mathbf{u}_c = 0$. Therefore, the optimal control policy for the above optimal control problem can be obtained as

$$\mathbf{u}_c^* = -\frac{1}{2} \mathbf{R}_1^{-1} \mathbf{B}_0^T \nabla J_1^* \quad (16)$$

Substituting the optimal control policy Eq. (16) and Eq. (14) into Eq. (15), the HJB equation can be expressed as

$$\mathbf{x}^T \mathbf{Q}_1 \mathbf{x} + (\nabla J_1^*)^T \mathbf{f}_0(\mathbf{x}) - \frac{1}{4} (\nabla J_1^*)^T \mathbf{B}_0 \mathbf{R}_1^{-1} \mathbf{B}_0^T \nabla J_1^* = 0, J_1^*(0) = 0 \quad (17)$$

In order to solve the optimal control policy, it is necessary to solve ∇J_1^* through the HJB equation represented by Eq. (17). However, directly solving the HJB equation is not only extremely challenging but also requires complete system model information, including $\mathbf{f}_0(\mathbf{x})$ and $\mathbf{B}_0$.

When solving the HJB equation, the values of the state variables are computed using the backward induction method. However, as the dimension of the state variables increases, the computational complexity grows exponentially, leading to what is known as the "curse of dimensionality." Therefore, in this paper, policy iteration algorithms are employed to solve the HJB equation.

Step 1: Initialization: Select an admissible control policy $\mathbf{u}_c^0$ as the initial iteration policy

Step 2: Policy evaluation: Update the estimated value of the cost function

$$J_1^i(\mathbf{x}(t)) = \int_t^{t+T} r_1(\mathbf{x}(\tau), \mathbf{u}_c^i(\tau)) d\tau + J_1^i(\mathbf{x}(t+T)), J_1^i(0) = 0 \quad (18)$$

where, i is the number of iterations, T is the sampling time interval.

Step 3: Policy improvement: Update control policy

$$\mathbf{u}_c^{i+1} = \min_{\mathbf{u}_c} H(\mathbf{x}, \mathbf{u}_c, \nabla J_1^i) \quad (19)$$

or

$$\mathbf{u}_c^{i+1} = -\frac{1}{2} \mathbf{R}_1^{-1} \mathbf{B}_0^T \nabla J_1^i \quad (20)$$

Step 4: End judgment: If the cost function finally converges, it ends; otherwise, go back to Step 2.

The above policy iteration algorithm begins with an initial admissible policy (stable policy) and converges to the optimal control policy through continuous iteration. The convergence proof of the algorithm can be found in [28]. It is necessary for the initial control policy to be admissible. If the initial control policy is not admissible, the system may diverge during the control policy update process, making algorithm convergence very difficult.

Compared to directly solving the optimal control policy through the HJB equation, the policy iteration algorithm is more feasible and does not require knowledge of the system's internal dynamic response. Given that the state and action spaces in the control problem are continuous and the state space has an infinite dimension, it is challenging to evaluate the cost function for all states using Eq. (18). Therefore, the reinforcement learning algorithm is introduced to approximate the cost functions via critic networks and to approximate optimal control policies through policy iteration.

3.1.2. Approximate optimal controller based on reinforcement learning

Given the powerful nonlinear approximation capability of neural networks, the critic network is employed to approximate the optimal cost function J_1^* as

$$J_1^* = \mathbf{W}_{c1}^T \sigma_{c1}(\mathbf{x}) + \varepsilon_{c1} \quad (21)$$

where, $\mathbf{x}$ refer to the inputs of the critic network, $\mathbf{W}_{c1}$ refer to the optimal critic network parameters, σ_{c1} refer to the activation functions of the critic network, ε_{c1} is the approximation error of the critic network.

Therefore, the partial derivative of the optimal cost function with respect to the state variables can be expressed as

$$\nabla J_1^* = (\nabla \sigma_{c1}(\mathbf{x}))^T \mathbf{W}_{c1} + \nabla \varepsilon_{c1} \quad (22)$$

where, $\nabla \sigma_{c1}(\mathbf{x}) = \partial \sigma_{c1}(\mathbf{x}) / \partial \mathbf{x}$ and $\nabla \varepsilon_{c1} = \partial \varepsilon_{c1} / \partial \mathbf{x}$ are the partial derivatives of the activation functions and approximation error of the critic network with respect to the state variables.

Using a neural network to directly obtain the co-state ∇J_1^* is another method to approximate the optimal control law. However, the convergence rate is slower because the gradient is more sensitive to noise and approximation errors.

According to Eq. (22), the optimal control policy can be expressed by

$$\mathbf{u}_c^* = -\frac{1}{2} \mathbf{R}_1^{-1} \mathbf{B}_0^T ((\nabla \sigma_{c1}(\mathbf{x}))^T \mathbf{W}_{c1} + \nabla \varepsilon_{c1}) \quad (23)$$

The optimal critic network parameters and the approximation error of the critic network are unknown, so the estimated value of the cost function $\hat{J}_1$ can be expressed as

$$\hat{J}_1 = \widehat{\mathbf{W}}_{c1}^T \sigma_{c1}(\mathbf{x}) \quad (24)$$

where, $\widehat{\mathbf{W}}_{c1}$ are the estimated values of the critic network parameters $\mathbf{W}_{c1}$.

Therefore, the partial derivative of the estimated value of the cost function with respect to the state variables can be expressed as

$$\nabla \hat{J}_1 = (\nabla \sigma_{c1}(\mathbf{x}))^T \widehat{\mathbf{W}}_{c1} \quad (25)$$

Using the critic network to approximate the cost function, the policy improvement algorithm as Eq. (18) can be rewritten as

$$\widehat{\mathbf{W}}_{c1}^{iT} \sigma_{c1}(\mathbf{x}(t)) = \int_t^{t+T} r_1(\mathbf{x}(\tau), \mathbf{u}_c^i(\tau)) d\tau + \widehat{\mathbf{W}}_{c1}^{iT} \sigma_{c1}(\mathbf{x}(t+T)) \quad (26)$$

Since there is an approximation error in the critic network, the temporal difference error is defined as follows

$$e_{c1}^i = \int_t^{t+T} r_1(\mathbf{x}(\tau), \mathbf{u}_c^i(\tau)) d\tau + \widehat{\mathbf{W}}_{c1}^{iT} [\sigma_{c1}(\mathbf{x}(t+T)) - \sigma_{c1}(\mathbf{x}(t))] \quad (27)$$

The ultimate goal of the critic network is to approximate the optimal cost function as closely as possible. Ideally, the temporal difference error should be zero. However, due to approximation errors in the critic network, it is challenging to ensure that the temporal difference error is exactly zero.

Therefore, to optimize and solve for the critic network parameters, the optimization objective can be set as the outer product of the

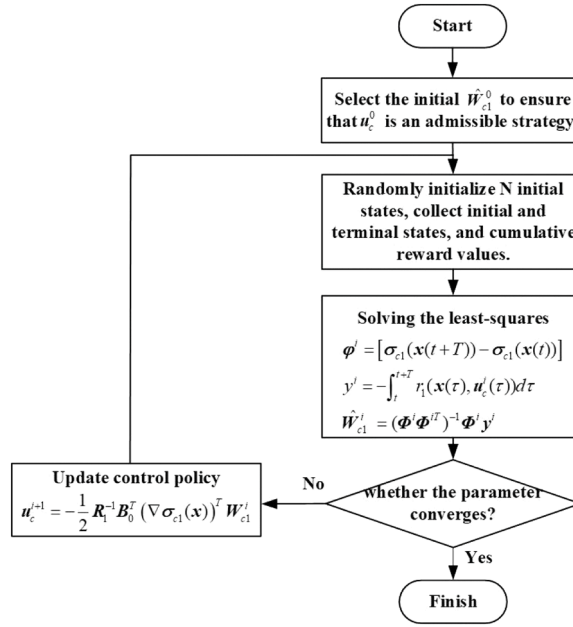


Fig. 1. Offline-based reinforcement learning-based approximate optimal controller.

temporal difference errors as follows

$$\min_{\hat{W}_{c1}} \frac{1}{2} e_{c1}^i e_{c1}^i \quad (28)$$

Since the temporal difference error e_{c1}^i is a scalar, a set of data can only construct one equation through Eq. (27). However, in order to reduce the approximation error of the critic network, the dimension of parameters of the critic network is usually relatively large. In this way, optimizing and solving according to Eq. (28) will result in non-uniqueness of estimated values of critic network parameters.

To address the aforementioned issue, we first generate numerous distinct state trajectories using different initial states for the same iteration process in an offline manner. By calculating the initial and terminal states of each trajectory, as well as the accumulated reward values, we obtain multiple different temporal difference errors through Eq. (27). Provided that there is a sufficient amount of data, optimization and solving based on Eq. (28) can ensure the uniqueness of the critic network parameters.

Defining $\phi = [\sigma_{c1}(x(t+T)) - \sigma_{c1}(x(t))]$, $y^i = -\int_t^{t+T} r_1(x(\tau), u_c^i(\tau)) d\tau$, the temporal difference error can be expressed as

$$e_{c1}^i = \hat{W}_{c1}^{iT} \phi - y^i = \phi^T \hat{W}_{c1}^i - y^i \quad (29)$$

The analytical solution of the optimization problem, as formulated in Eq. (28), can be obtained using the least squares algorithm. Therefore, in the i th iteration process, the estimated values of the critic network parameters $\hat{W}_{c1}^i$ can be updated by the follows

$$\hat{W}_{c1}^i = (\Phi \Phi^T)^{-1} \Phi y^i \quad (30)$$

where, $\Phi = [\phi_1 \ \phi_2 \ \dots \ \phi_k]$, and $\phi_i (i=1 \dots k)$ is the result of the ϕ in the i th iteration. y^i is the column vector composed of y^j . By collecting control inputs, system states and reward data, the critic network parameters are updated through Eq. (30).

Based on the estimation of the critic network parameters, a new control policy is updated according to policy improvement

$$u_c^{i+1} = -\frac{1}{2} R_1^{-1} B_0^T (\nabla \sigma_{c1}(x))^T \hat{W}_{c1}^i \quad (31)$$

It is worth mentioning that although the control policy (31) is an explicit expression, it can still be regarded as an action network, where the activation function of the action network is the partial derivative of the activation function of the critic network with respect to state variables. However, under the framework of optimal control, if the reward function is in the form shown in Eq. (12), then the update of the action network has the above analytical solution and does not need to be solved by optimization.

Fig. 1 shows the designing process of offline-based reinforcement learning-based approximate optimal controller. The control policy is used to generate training data, and the least squares algorithm is used to update the evaluation network parameters $\hat{W}_{c1}^i$, so as to update the control policy u_c^{i+1} . Then the new control policy is used to repeat the above steps until the evaluation network parameters converge, and the approximate optimal control policy is obtained.

Since the reinforcement learning algorithm described above is essentially a policy iteration algorithm, the selection of the initial

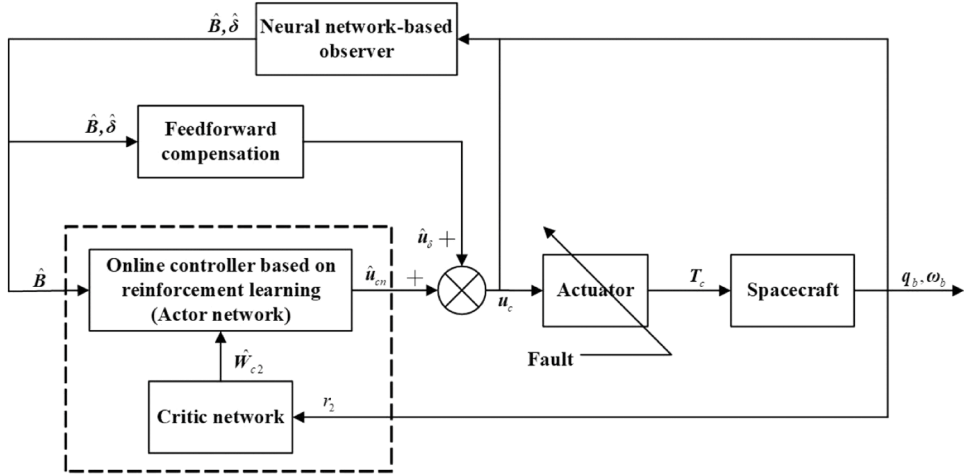


Fig. 2. The structure of an online fault-tolerant control system based on reinforcement learning.

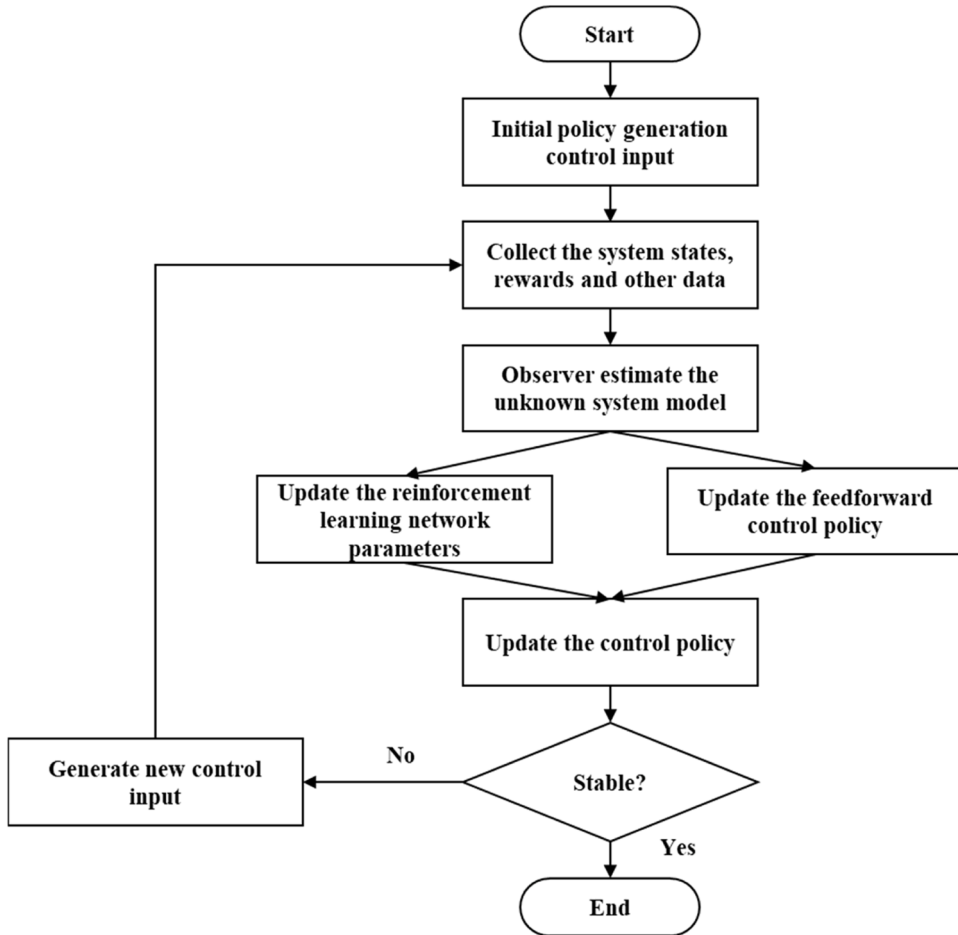


Fig. 3. The flow of the fault-tolerant controller.

policy is critical. Specifically, the initial control policy must be admissible to ensure that subsequent iterative control policies are also admissible and can gradually converge to the optimal control policy. In practical applications, an initial admissible policy can be chosen as a PID control policy or a feedback control policy.

3.2. Online fault-tolerant controller based on reinforcement learning

The online fault-tolerant control system based on reinforcement learning consists of three parts: (1) a neural network-based observer; (2) an online reinforcement learning controller; (3) a feedforward compensation algorithm. The neural network-based observer estimates unknown input matrix $\mathbf{B}$ and partial disturbances δ which include additive faults, disturbance torques and a part of rotational inertia effects. Using the estimated input matrix and the critic network parameters updated by the least squares algorithm, an attitude stabilization controller based on reinforcement learning is developed, which is easily implementable online. Finally, a feedforward compensation algorithm based on the neural network is incorporated into the reinforcement learning controller to address the partial disturbances and achieve online fault-tolerant control. Fig. 2 shows the structure of the entire online fault-tolerant control system based on reinforcement learning.

Fig. 3 shows the flow of the fault-tolerant controller. The entire process can be described as follows: First, the initial control policy is used to generate control inputs that act on the system, and then data such as the system's states and rewards are collected. Next, an observer is employed to estimate the system model, thereby updating the reinforcement learning and feedforward control strategies. If the system achieves stable results, the process concludes; otherwise, a new control policy is used to generate new control inputs, and the above steps are repeated.

3.2.1. Neural network-based observer

When the spacecraft is subject to rotational inertia uncertainty, actuator faults, and unknown external disturbances, both the system input matrix $\mathbf{B}$ and the partial disturbances δ will change. Since the update of the control policy relies on the information of the input matrix $\mathbf{B}$ and cannot handle the adverse effects of the partial disturbances δ , a neural network-based observer has been designed. The neural network is employed to approximate the unknown system dynamics $\mathbf{f}(\mathbf{x})$. Meanwhile, the observer provides data to update the network parameters for a more accurate approximation, while estimating the input matrix $\mathbf{B}$ and the partial disturbances δ .

Using neural networks to approximate nonlinear functions

$$\mathbf{f}(\mathbf{x}) = \mathbf{W}^T \sigma(\mathbf{x}) + \epsilon \quad (32)$$

where, $\mathbf{W}$ is the ideal weight, $\sigma(\mathbf{x})$ refers to a properly constructed activation function, ϵ refers to the neural network approximation error, and its norm has an upper bound ϵ , that is $\|\epsilon\| < \epsilon$.

Therefore, Eq. (8) can be expressed as

$$\dot{\mathbf{x}} = \mathbf{W}^T \sigma(\mathbf{x}) + \mathbf{B} \mathbf{u}_c + \delta + \epsilon \quad (33)$$

In order to obtain the system input matrix $\mathbf{B}$ and the partial disturbances δ , an observer is constructed as follows

$$\dot{\hat{\mathbf{x}}} = \hat{\mathbf{W}}^T \sigma(\hat{\mathbf{x}}) + \hat{\mathbf{B}} \mathbf{u}_c + \hat{\delta} + \mathbf{K}_o (\mathbf{x} - \hat{\mathbf{x}}) \quad (34)$$

where, $\hat{\mathbf{x}}$ is the estimated values of state $\mathbf{x}$, $\hat{\mathbf{W}}$ is the estimated value of $\mathbf{W}$, $\hat{\mathbf{B}}$ is the estimated value of the input matrix $\mathbf{B}$, $\hat{\delta}$ is the estimated value of the partial disturbances, $\mathbf{K}_o$ is a positive definite observation gain matrix.

The update laws for the above parameters are

$$\dot{\hat{\mathbf{W}}} = -\alpha_w (\sigma(\hat{\mathbf{x}}) \hat{\mathbf{x}}^T + k_w \hat{\mathbf{W}}) \quad (35)$$

$$\dot{\hat{\mathbf{B}}} = -\alpha_b (\tilde{\mathbf{x}} \mathbf{u}_c^T + k_b \hat{\mathbf{B}}) \quad (36)$$

$$\dot{\hat{\delta}} = -\alpha_\delta (\tilde{\mathbf{x}} + k_\delta \hat{\delta}) \quad (37)$$

where, $\tilde{\mathbf{x}} = \hat{\mathbf{x}} - \mathbf{x}$ denoted the observer estimation error, $\alpha_w, \alpha_b, \alpha_\delta, k_w, k_b, k_\delta$ are all constants greater than zero.

Based on Eq. (33) and Eq. (34), the state equation of the observer error can be expressed as

$$\dot{\tilde{\mathbf{x}}} = -\mathbf{K}_o \tilde{\mathbf{x}} + \tilde{\mathbf{W}}^T \sigma(\hat{\mathbf{x}}) + \tilde{\mathbf{B}} \mathbf{u}_c + \tilde{\delta} + \mathbf{W}^T \sigma(\mathbf{x}, \hat{\mathbf{x}}) - \epsilon \quad (38)$$

where, $\tilde{\mathbf{W}} = \hat{\mathbf{W}} - \mathbf{W}$, $\tilde{\mathbf{B}} = \hat{\mathbf{B}} - \mathbf{B}$ and $\tilde{\delta} = \hat{\delta} - \delta$ are parameter estimation errors, $\tilde{\sigma}(\mathbf{x}, \hat{\mathbf{x}}) = \sigma(\hat{\mathbf{x}}) - \sigma(\mathbf{x})$ denotes the activation function observation error.

Theorem 1: For the unknown nonlinear system described in Eq. (8), using a neural network-based observer defined in Eq. (34) and the parameter update laws given in Eqs. (35) ~ (37), it can be guaranteed that the estimation error is bounded.

Proof: Define the following Lyapunov function

$$V_1 = \frac{1}{2} \tilde{\mathbf{x}}^T \tilde{\mathbf{x}} + \frac{1}{2\alpha_w} \text{tr}(\tilde{\mathbf{W}}^T \tilde{\mathbf{W}}) + \frac{1}{2\alpha_b} \text{tr}(\tilde{\mathbf{B}}^T \tilde{\mathbf{B}}) + \frac{1}{2\alpha_\delta} \tilde{\delta}^T \tilde{\delta} \quad (39)$$

Taking the time derivative of V_1 and substituting Eqs. (34)-(37) into $\dot{V}_1$, $\dot{V}_1$ can be derived as

$$\begin{aligned}
\dot{V}_1 &= \tilde{\mathbf{x}}^T \dot{\tilde{\mathbf{x}}} + \frac{1}{\alpha_w} \text{tr}(\tilde{\mathbf{W}}^T \dot{\tilde{\mathbf{W}}}) + \frac{1}{\alpha_b} \text{tr}(\tilde{\mathbf{B}}^T \dot{\tilde{\mathbf{B}}}) + \frac{1}{\alpha_\delta} \tilde{\delta}^T \dot{\tilde{\delta}} \\
&= \tilde{\mathbf{x}}^T (-\mathbf{K}_o \tilde{\mathbf{x}} + \tilde{\mathbf{W}}^T \boldsymbol{\sigma}(\hat{\mathbf{x}}) + \tilde{\mathbf{B}} \mathbf{u} + \tilde{\delta} + \mathbf{W}^T \tilde{\boldsymbol{\sigma}}(\mathbf{x}, \hat{\mathbf{x}}) - \boldsymbol{\varepsilon}) \\
&\quad - \text{tr}(\tilde{\mathbf{W}}^T \boldsymbol{\sigma}(\hat{\mathbf{x}}) \tilde{\mathbf{x}}^T + k_w \tilde{\mathbf{W}}^T \tilde{\mathbf{W}}) - \text{tr}(\tilde{\mathbf{B}}^T \tilde{\mathbf{x}} \mathbf{u}_c^T + k_b \tilde{\mathbf{B}}^T \tilde{\mathbf{B}}) \\
&\quad - (\tilde{\delta}^T \tilde{\mathbf{x}} + k_\delta \tilde{\delta}^T \tilde{\delta})
\end{aligned} \tag{40}$$

since $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$, and when $\mathbf{BA}$ is a scalar, $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA}) = \mathbf{BA}$ holds. Therefore, Eq. (40) can be simplified as

$$\dot{V}_1 = -\tilde{\mathbf{x}}^T \mathbf{K}_o \tilde{\mathbf{x}} + \tilde{\mathbf{x}}^T \mathbf{w}_\sigma - k_w \text{tr}(\tilde{\mathbf{W}}^T \tilde{\mathbf{W}}) - k_b \text{tr}(\tilde{\mathbf{B}}^T \tilde{\mathbf{B}}) - k_\delta \tilde{\delta}^T \tilde{\delta} \tag{41}$$

where, $\mathbf{w}_\sigma = \mathbf{W}^T \tilde{\boldsymbol{\sigma}}(\mathbf{x}, \hat{\mathbf{x}}) - \boldsymbol{\varepsilon}$.

Applying Young's inequality yields

$$\begin{aligned}
\dot{V}_1 &\leq -\lambda_{\min}(\mathbf{K}_o) \tilde{\mathbf{x}}^T \tilde{\mathbf{x}} + \frac{1}{2} \tilde{\mathbf{x}}^T \tilde{\mathbf{x}} + \frac{1}{2} \mathbf{w}_\sigma^T \mathbf{w}_\sigma - \frac{1}{2} k_w \text{tr}(\tilde{\mathbf{W}}^T \tilde{\mathbf{W}}) + \frac{1}{2} k_w \text{tr}(\mathbf{W}^T \mathbf{W}) \\
&\quad - \frac{1}{2} k_b \text{tr}(\tilde{\mathbf{B}}^T \tilde{\mathbf{B}}) + \frac{1}{2} k_b \text{tr}(\mathbf{B}^T \mathbf{B}) - \frac{1}{2} k_\delta \tilde{\delta}^T \tilde{\delta} + \frac{1}{2} k_\delta \delta^T \delta
\end{aligned} \tag{42}$$

By combining the same coefficients, Eq. (42) can be rewritten as

$$\dot{V}_1 \leq \lambda_1 V_1 + \mu_1 \tag{43}$$

where

$$\lambda_1 = \min\{2\lambda_{\min}(\mathbf{K}_o) - 1, \alpha_w k_w, \alpha_b k_b, \alpha_\delta k_\delta\} \tag{44}$$

$$\mu_1 = \frac{1}{2} \mathbf{w}_\sigma^T \mathbf{w}_\sigma + \frac{1}{2} k_w \text{tr}(\mathbf{W}^T \mathbf{W}) + \frac{1}{2} k_b \text{tr}(\mathbf{B}^T \mathbf{B}) + \frac{1}{2} k_\delta \delta^T \delta \tag{45}$$

Integrating Eq. (43) yields

$$V_1(t) \leq \frac{\mu_1}{\lambda_1} + \left(V_1(0) - \frac{\mu_1}{\lambda_1} \right) e^{-\lambda_1 t} \tag{46}$$

where, $V_1(0)$ is the initial value of Lyapunov function V_1 .

From Eq. (46), it can be seen that the Lyapunov function V_1 is bounded. Furthermore, according to theorem 1, it follows that the observer estimation error $\tilde{\mathbf{x}}$ and the parameter estimation errors $\tilde{\mathbf{W}}, \tilde{\mathbf{B}}, \tilde{\delta}$ are all bounded. This completes the proof.

When k_w, k_b and k_δ are determined, μ_1 has an upper bound. From Eq. (46), it can be seen that the upper bound of Lyapunov function V_1 depends on μ_1 / λ_1 . By increasing the values of $\mathbf{K}_o, \alpha_w, \alpha_b$, and α_δ , the value of λ_1 can be increased, thereby reducing the upper bound of V_1 and, consequently, decreasing the estimation error of the observer.

In particular, when the parameters k_w, k_b and k_δ are set to zero, the following inequality holds

$$\dot{V}_1 \leq -\lambda_{\min}(\mathbf{K}_o) \tilde{\mathbf{x}}^T \tilde{\mathbf{x}} + \frac{1}{2} \tilde{\mathbf{x}}^T \tilde{\mathbf{x}} + \frac{1}{2} \mathbf{w}_\sigma^T \mathbf{w}_\sigma \tag{47}$$

Observer estimation error converges to set G_1

$$G_1 = \left\{ \tilde{\mathbf{x}} \mid \|\tilde{\mathbf{x}}\| \leq \frac{\|\mathbf{w}_\sigma\|}{2\lambda_{\min}(\mathbf{K}_o) - 1} \right\}$$

Therefore, if $\lambda_{\min}(\mathbf{K}_o) > 0.5$ holds, the observer estimation error is bounded.

3.2.2. Online controller based on reinforcement learning

To achieve stable attitude control of the spacecraft in the presence of model uncertainties and actuator faults, a fault-tolerant controller is designed using the reinforcement learning method. Similar to the above offline reinforcement learning algorithm, define the following reward function

$$r_2(\mathbf{x}, \mathbf{u}_{cn}) = \mathbf{x}^T \mathbf{Q}_2 \mathbf{x} + \mathbf{u}_{cn}^T \mathbf{R}_2 \mathbf{u}_{cn} \tag{48}$$

where, $\mathbf{Q}_2$ and $\mathbf{R}_2$ are positive-definite matrices with appropriate dimensions, representing the weights of control performance and control input, $\mathbf{u}_{cn}$ is the control policy by online reinforcement learning.

Define the following cost function

$$J_2(t) = \int_t^\infty r_2(\tau) d\tau \tag{49}$$

Using the critic network to approximate the optimal cost function as follows

$$J_2^* = \mathbf{W}_{c2}^T \sigma_{c2}(\mathbf{x}) + \varepsilon_{c2} \quad (50)$$

where, $\mathbf{x}$ refer to the inputs of the critic network, $\mathbf{W}_{c2}$ refer to the optimal critic network parameters, σ_{c2} refer to the activation functions of the critic network, ε_{c2} is the approximation error of the critic network.

The optimal critic network parameters and the approximation error of the critic network are unknown, so the estimated value of the cost function $\hat{J}_2$ can be expressed as

$$\hat{J}_2 = \hat{\mathbf{W}}_{c2}^T \sigma_{c2}(\mathbf{x}) \quad (51)$$

where, $\hat{\mathbf{W}}_{c2}$ are the estimated values of the critic network parameters $\mathbf{W}_{c2}$.

Define the temporal difference error in the same form as Eq. (27)

$$e_{c2} = \int_t^{t+T} r_2(\mathbf{x}(\tau), \mathbf{u}_{cn}(\tau)) d\tau + \hat{\mathbf{W}}_{c2}^T [\sigma_{c2}(\mathbf{x}(t+T)) - \sigma_{c2}(\mathbf{x}(t))] \quad (52)$$

Due to the real-time requirements of online fault-tolerant control, the RLS method is employed to update the parameters of the critic network.

Letting $\varphi_{c2} = \sigma_{c2}(\mathbf{x}(t+T)) - \sigma_{c2}(\mathbf{x}(t))$, $y_{c2} = -\int_t^{t+T} r_2(\mathbf{x}(\tau), \mathbf{u}_{cn}(\tau)) d\tau$, then $\hat{\mathbf{W}}_{c2}$ can be updated by RLS method

$$\hat{\mathbf{W}}_{c2}(k+1) = \hat{\mathbf{W}}_{c2}(k) + \mathbf{K}(k+1) [y_{c2}(k+1) - \varphi_{c2}^T(k+1) \hat{\mathbf{W}}_{c2}(k)] \quad (53)$$

$$\mathbf{K}(k+1) = \mathbf{P}(k) \varphi_{c2}(k+1) / [1 + \varphi_{c2}^T(k+1) \mathbf{P}(k) \varphi_{c2}(k+1)] \quad (54)$$

$$\mathbf{P}(k+1) = [\mathbf{I}_6 - \mathbf{K}(k+1) \varphi_{c2}^T(k+1)] \mathbf{P}(k) \quad (55)$$

where, k refers to the sequence number of the collected data, $\mathbf{P}$ is the covariance matrix. By collecting control inputs, system states, and reward data, the parameters are updated using Eq. (53).

After determining the initial values of $\hat{\mathbf{W}}_{c2}$ and $\mathbf{P}$, the critic network parameters can be updated based on φ_{c2} and y_{c2} collected. There are two methods for selecting the initial values: One method is to set $\mathbf{P}(0)$ as a sufficiently large unit matrix and select $\hat{\mathbf{W}}_{c2}(0)$ as a sufficiently small real vector or a zero vector; the other method involves using an offline nominal controller based on offline reinforcement learning, where the trained $\hat{\mathbf{W}}_{c2}$ and corresponding $\mathbf{P}$ are used as the initial values. To ensure that the initial control policy is admissible, the second method is adopted in this study.

Compared to the least squares algorithm, the RLS method offers several advantages for updating critic network parameters. Specifically, RLS allows for parameter updates at each sampling time without needing to wait for sufficient data accumulation. This feature enhances real-time performance and is more suitable for online implementation. Additionally, the shorter update intervals result in smoother parameter changes and reduce the likelihood of abrupt variations in control inputs. Moreover, RLS avoids matrix inversion, thereby effectively reducing computational complexity.

Based on the above analysis, the optimal online reinforcement learning control policy can be expressed by

$$\mathbf{u}_{cn}^* = -\frac{1}{2} \mathbf{R}_2^{-1} \mathbf{B}^T (\nabla \sigma_{c2}(\mathbf{x}))^T \mathbf{W}_{c2} \quad (56)$$

Since the actual values of $\mathbf{B}$ and $\mathbf{W}_{c2}$ are unknown, $\hat{\mathbf{B}}$ and $\hat{\mathbf{W}}_{c2}$ are used for approximation. Therefore, the online reinforcement learning approximate optimal controller can be expressed as

$$\hat{\mathbf{u}}_{cn} = -\frac{1}{2} \mathbf{R}_2^{-1} \hat{\mathbf{B}}^T (\nabla \sigma_{c2}(\mathbf{x}))^T \hat{\mathbf{W}}_{c2} \quad (57)$$

It can be seen from Eq. (9) that $\mathbf{B}$ is affected by the multiplicative actuator faults. By using the observer based on neural network to estimate $\mathbf{B}$ and combining reinforcement learning method to design the controller, the control objective can be achieved and the adverse effects due to multiplicative faults can be eliminated.

3.2.3. Feedforward compensation algorithm

When rotational inertia uncertainty, additive actuator faults and disturbance torques exist, the partial disturbances δ in Eq. (8) is non-zero and independent of the system states. This will make it difficult to ensure that the system states converge to zero using only the online reinforcement learning controller proposed in Section 3.2.2. Therefore, a feedforward compensation algorithm is added to the online reinforcement learning controller to deal with the impact of the partial disturbances δ .

The feedforward compensation control law can be given by

$$\hat{\mathbf{u}}_{\delta} = -\hat{\mathbf{B}}^+ \hat{\delta} \quad (58)$$

where, $\hat{\delta}$ is given by the neural network-based observer.

Finally, the command control torque of the actuator is described by

$$\mathbf{u}_c = \hat{\mathbf{u}}_{cn} + \hat{\mathbf{u}}_\delta \quad (59)$$

It can be seen from Eq. (59) that the online fault-tolerant controller based on reinforcement learning can directly calculate the command control torque of the actuator. The entire fault-tolerant control scheme is divided into three parts. Firstly, a neural network-based observer is designed to estimate the unknown system parameters, including the input matrix $\mathbf{B}$ and the partial disturbances δ . Secondly, with the estimated value of the input matrix $\mathbf{B}$, a reinforcement learning based online controller is proposed to achieve attitude stabilization of spacecraft, while reducing the impact of multiplicative faults of the actuator. Finally, a feedforward compensation algorithm is added to the attitude controller to eliminate the adverse effects of additive faults of actuator and external disturbance torques.

3.2.4. Controller stability analysis

Theorem 2: For the system model with inertia uncertainty, actuator faults, and disturbances as described in Eq. (8), the observer parameters defined in Eq. (34) are updated according to Eqs. (35)–(37), and the critic network parameters defined in Eq. (51) are updated according to Eqs. (53)–(55). Under these conditions, the online reinforcement learning-based fault-tolerant controller, composed of Eqs. (57) and (58) as shown in Eq. (59), can guarantee the bounded stability of the system state.

Proof: Define the following Lyapunov function

$$V_2 = \frac{1}{2} \mathbf{x}^T \mathbf{x} + J_2^* \quad (60)$$

Taking the time derivative of the above equation and substituting Eqs.(8) and (59) yields

$$\begin{aligned} \dot{V}_2 &= \mathbf{x}^T \dot{\mathbf{x}} + (\nabla J_2^*)^T \dot{\mathbf{x}} \\ &= \mathbf{x}^T (\mathbf{f}(\mathbf{x}) + \mathbf{B}(\hat{\mathbf{u}}_{cn} + \hat{\mathbf{u}}_\delta) + \delta) + (\nabla J_2^*)^T (\mathbf{f}(\mathbf{x}) + \mathbf{B}(\hat{\mathbf{u}}_{cn} + \hat{\mathbf{u}}_\delta) + \delta) \\ &= \mathbf{x}^T (\mathbf{f}(\mathbf{x}) + \mathbf{B}(\hat{\mathbf{u}}_{cn} + \tilde{\mathbf{u}}_\delta)) + (\nabla J_2^*)^T (\mathbf{f}(\mathbf{x}) + \mathbf{B}(\hat{\mathbf{u}}_{cn} + \tilde{\mathbf{u}}_\delta)) \end{aligned} \quad (61)$$

where, $\tilde{\mathbf{u}}_\delta = \hat{\mathbf{u}}_\delta + \mathbf{B}^+ \delta$ is the feedforward compensation error.

According to Eq. (59), it yields

$$(\nabla J_2^*)^T \mathbf{B} = \mathbf{W}_{c2}^T \nabla \sigma_{c2}(\mathbf{x}) \mathbf{B} = -2\mathbf{u}_{cn}^{*T} \mathbf{R}_2 \quad (62)$$

Due to

$$\mathbf{x}^T \mathbf{Q}_2 \mathbf{x} + \mathbf{u}_{cn}^{*T} \mathbf{R}_2 \mathbf{u}_{cn}^* + (\nabla J_2^*)^T (\mathbf{f}(\mathbf{x}) + \mathbf{B} \mathbf{u}_{cn}^*) = 0 \quad (63)$$

Eq. (61) can be expressed as

$$\begin{aligned} \dot{V}_2 &= \mathbf{x}^T (\mathbf{f}(\mathbf{x}) + \mathbf{B}(\hat{\mathbf{u}}_{cn} + \tilde{\mathbf{u}}_\delta)) + (\nabla J_2^*)^T \mathbf{f}(\mathbf{x}) + (\nabla J_2^*)^T \mathbf{B}(\hat{\mathbf{u}}_{cn} + \tilde{\mathbf{u}}_\delta) \\ &= \mathbf{x}^T (\mathbf{f}(\mathbf{x}) + \mathbf{B}(\hat{\mathbf{u}}_{cn} + \tilde{\mathbf{u}}_\delta)) - \mathbf{x}^T \mathbf{Q}_2 \mathbf{x} - \mathbf{u}_{cn}^{*T} \mathbf{R}_2 \mathbf{u}_{cn}^* - (\nabla J_2^*)^T \mathbf{B} \mathbf{u}_{cn}^* \\ &\quad + (\nabla J_2^*)^T \mathbf{B} \hat{\mathbf{u}}_{cn} + (\nabla J_2^*)^T \mathbf{B} \tilde{\mathbf{u}}_\delta \\ &= \mathbf{x}^T (\mathbf{f}(\mathbf{x}) + \mathbf{B}(\hat{\mathbf{u}}_{cn} + \tilde{\mathbf{u}}_\delta)) - \mathbf{x}^T \mathbf{Q}_2 \mathbf{x} + \mathbf{u}_{cn}^{*T} \mathbf{R}_2 \mathbf{u}_{cn}^* - 2\mathbf{u}_{cn}^{*T} \mathbf{R}_2 \hat{\mathbf{u}}_{cn} - 2\mathbf{u}_{cn}^{*T} \mathbf{R}_2 \tilde{\mathbf{u}}_\delta \\ &= \mathbf{x}^T (\mathbf{f}(\mathbf{x}) + \mathbf{B}(\hat{\mathbf{u}}_{cn} + \tilde{\mathbf{u}}_\delta)) - \mathbf{x}^T \mathbf{Q}_2 \mathbf{x} + \tilde{\mathbf{u}}_{cn}^T \mathbf{R}_2 \tilde{\mathbf{u}}_{cn} - \hat{\mathbf{u}}_{cn}^T \mathbf{R}_2 \hat{\mathbf{u}}_{cn} - 2\mathbf{u}_{cn}^{*T} \mathbf{R}_2 \tilde{\mathbf{u}}_\delta \end{aligned} \quad (64)$$

where, $\tilde{\mathbf{u}}_{cn} = \hat{\mathbf{u}}_{cn} - \mathbf{u}_{cn}^*$ is the estimation error of the optimal control policy.

Since $-2\mathbf{u}_{cn}^{*T} \mathbf{R}_2 \tilde{\mathbf{u}}_\delta \leq \mathbf{u}_{cn}^{*T} \mathbf{R}_2 \mathbf{u}_{cn}^* + \tilde{\mathbf{u}}_\delta^T \mathbf{R}_2 \tilde{\mathbf{u}}_\delta$, $\dot{V}_2$ satisfies the following inequality

$$\begin{aligned} \dot{V}_2 &= \mathbf{x}^T (\mathbf{f}(\mathbf{x}) + \mathbf{B}(\hat{\mathbf{u}}_{cn} + \tilde{\mathbf{u}}_\delta)) - \mathbf{x}^T \mathbf{Q}_2 \mathbf{x} + \tilde{\mathbf{u}}_{cn}^T \mathbf{R}_2 \tilde{\mathbf{u}}_{cn} - \hat{\mathbf{u}}_{cn}^T \mathbf{R}_2 \hat{\mathbf{u}}_{cn} - 2\mathbf{u}_{cn}^{*T} \mathbf{R}_2 \tilde{\mathbf{u}}_\delta \\ &\leq \mathbf{x}^T (\mathbf{f}(\mathbf{x}) + \mathbf{B}(\hat{\mathbf{u}}_{cn} + \tilde{\mathbf{u}}_\delta)) - \mathbf{x}^T \mathbf{Q}_2 \mathbf{x} + \tilde{\mathbf{u}}_{cn}^T \mathbf{R}_2 \tilde{\mathbf{u}}_{cn} - \hat{\mathbf{u}}_{cn}^T \mathbf{R}_2 \hat{\mathbf{u}}_{cn} + \mathbf{u}_{cn}^{*T} \mathbf{R}_2 \mathbf{u}_{cn}^* + \tilde{\mathbf{u}}_\delta^T \mathbf{R}_2 \tilde{\mathbf{u}}_\delta \end{aligned} \quad (65)$$

Since $\mathbf{f}(\mathbf{x})$ satisfies Lipschitz conditions, there is $\|\mathbf{f}(\mathbf{x})\| \leq D_f \|\mathbf{x}\|$, where D_f is a positive real number. Combining with Young's inequality, the following inequality holds

$$\begin{aligned} \dot{V}_2 &\leq \mathbf{x}^T (\mathbf{f}(\mathbf{x}) + \mathbf{B}(\hat{\mathbf{u}}_{cn} + \tilde{\mathbf{u}}_\delta)) - \mathbf{x}^T \mathbf{Q}_2 \mathbf{x} + \tilde{\mathbf{u}}_{cn}^T \mathbf{R}_2 \tilde{\mathbf{u}}_{cn} - \hat{\mathbf{u}}_{cn}^T \mathbf{R}_2 \hat{\mathbf{u}}_{cn} + \mathbf{u}_{cn}^{*T} \mathbf{R}_2 \mathbf{u}_{cn}^* + \tilde{\mathbf{u}}_\delta^T \mathbf{R}_2 \tilde{\mathbf{u}}_\delta \\ &\leq D_f \mathbf{x}^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \mathbf{x} + \frac{1}{2} \tilde{\mathbf{u}}_{cn}^T \mathbf{B}^T \mathbf{B} \hat{\mathbf{u}}_{cn} + \frac{1}{2} \mathbf{x}^T \mathbf{x} + \frac{1}{2} \tilde{\mathbf{u}}_\delta^T \mathbf{B}^T \mathbf{B} \tilde{\mathbf{u}}_\delta \\ &\quad - \mathbf{x}^T \mathbf{Q}_2 \mathbf{x} + \tilde{\mathbf{u}}_{cn}^T \mathbf{R}_2 \tilde{\mathbf{u}}_{cn} - \hat{\mathbf{u}}_{cn}^T \mathbf{R}_2 \hat{\mathbf{u}}_{cn} + \mathbf{u}_{cn}^{*T} \mathbf{R}_2 \mathbf{u}_{cn}^* + \tilde{\mathbf{u}}_\delta^T \mathbf{R}_2 \tilde{\mathbf{u}}_\delta \\ &\leq -(\lambda_{\min}(\mathbf{Q}_2) - D_f - 1) \mathbf{x}^T \mathbf{x} - (\lambda_{\min}(\mathbf{R}_2) - 0.5\lambda_{\max}(\mathbf{B}^T \mathbf{B})) \hat{\mathbf{u}}_{cn}^T \hat{\mathbf{u}}_{cn} \\ &\quad + 0.5\lambda_{\max}(\mathbf{B}^T \mathbf{B}) \tilde{\mathbf{u}}_\delta^T \tilde{\mathbf{u}}_\delta + \lambda_{\max}(\mathbf{R}_2) (\tilde{\mathbf{u}}_{cn}^T \tilde{\mathbf{u}}_{cn} + \mathbf{u}_{cn}^{*T} \mathbf{u}_{cn}^* + \tilde{\mathbf{u}}_\delta^T \tilde{\mathbf{u}}_\delta) \end{aligned} \quad (66)$$

The Eq. (66) can be rewritten as

$$\dot{V}_2 \leq -\lambda_x \mathbf{x}^T \mathbf{x} - \lambda_{cn} \hat{\mathbf{u}}_{cn}^T \hat{\mathbf{u}}_{cn} + \mu_2 \quad (67)$$

where, λ_x, λ_{cn} and μ_2 are respectively expressed by

$$\lambda_x = \lambda_{\min}(\mathbf{Q}_2) - D_f - 1 \quad (68)$$

$$\lambda_{cn} = \lambda_{\min}(\mathbf{R}_2) - 0.5\lambda_{\max}(\mathbf{B}^T \mathbf{B}) \quad (69)$$

$$\mu_2 = 0.5\lambda_{\max}(\mathbf{B}^T \mathbf{B}) \tilde{\mathbf{u}}_\delta^T \tilde{\mathbf{u}}_\delta + \lambda_{\max}(\mathbf{R}_2) (\tilde{\mathbf{u}}_{cn}^T \tilde{\mathbf{u}}_{cn} + \mathbf{u}_{cn}^{*T} \mathbf{u}_{cn}^* + \tilde{\mathbf{u}}_\delta^T \tilde{\mathbf{u}}_\delta) \quad (70)$$

When $\lambda_{\min}(\mathbf{Q}_2) > D_f + 1$ and $\lambda_{\min}(\mathbf{R}_2) > 0.5\lambda_{\max}(\mathbf{B}^T \mathbf{B})$, both λ_x and λ_{cn} are greater than zero. According to Lyapunov Stability Theorem, the system state variables $\mathbf{x}$ are bounded and stable. In practical applications, the upper limits of D_f and $\lambda_{\max}(\mathbf{B}^T \mathbf{B})$ can be determined by considering the constraints imposed on the system, and then $\mathbf{Q}_2$ and $\mathbf{R}_2$ can be set accordingly.

This completes the proof.

Define the set G_2

$$G_2 = \left\{ \mathbf{x} \mid \|\mathbf{x}\| \leq \frac{\mu_2}{\lambda_x} \right\}$$

Under the premise of selecting appropriate parameters to ensure that λ_x and λ_{cn} are both greater than zero, if $\mathbf{x}$ does not belong to the set G_2 , then $\dot{V}_2 < 0$ holds. In this case, the value of the Lyapunov function will gradually decrease, and $\mathbf{x}$ will eventually converge to the set G_2 . Therefore, to reduce the control error of the system state variables, one can appropriately increase parameter λ_x or decrease parameter μ_2 . Specifically, while ensuring $\lambda_{\min}(\mathbf{Q}_2) > D_f + 1$ and $\lambda_{\min}(\mathbf{R}_2) > 0.5\lambda_{\max}(\mathbf{B}^T \mathbf{B})$, the eigenvalues of matrix $\mathbf{Q}_2$ can be increased appropriately or the eigenvalues of matrix $\mathbf{R}_2$ can be decreased appropriately.

4. Simulation results

In this section, a simulation example is carried out to illustrate the controller design procedures and verify the effectiveness of the proposed controller based on reinforcement learning.

The actual and nominal moments of inertia of the spacecraft are given respectively as

$$\mathbf{I}_b = \begin{bmatrix} 45.4 & 1.5 & 1 \\ 1.5 & 23.3 & 1.2 \\ 1 & 1.2 & 32.8 \end{bmatrix} \text{ kg} \cdot \text{m}^2, \mathbf{I}_{b0} = \begin{bmatrix} 40 & 0 & 0 \\ 0 & 20 & 0 \\ 0 & 0 & 28 \end{bmatrix} \text{ kg} \cdot \text{m}^2$$

The installation matrix of the actuator is given as

$$\mathbf{C} = \begin{bmatrix} 1 & 0 & 0 & 1/\sqrt{3} \\ 0 & 1 & 0 & 1/\sqrt{3} \\ 0 & 0 & 1 & 1/\sqrt{3} \end{bmatrix}$$

The initial attitude quaternion and angular velocity of the spacecraft are set by

$$\mathbf{Q}_b(0) = [0.9531 \ 0.1 \ 0.15 \ -0.12]^T \text{ and } \boldsymbol{\omega}_b(0) = [0.4 \ -0.2 \ 0.3]^T \text{ } ^\circ/\text{s}.$$

The disturbance torque acting on the spacecraft is given as

$$\mathbf{T}_d = \begin{bmatrix} 3\cos(0.001t) \\ \cos(0.001t) + 1 \\ -4\sin(0.001t) \end{bmatrix} \times 10^{-4} \text{ N} \cdot \text{m}$$

The actuator faults are set as: the x-axis actuator simultaneously fails at the 20th second, where the fault coefficient is $e_1 = 0.5$, and the additive fault is $\tilde{u}_{c1} = -0.1 \text{ N} \cdot \text{m}$.

The control parameters in this paper are categorized into four groups: neural network structure, observer parameters, reinforcement learning parameters, and RLS parameters. The neural network structure employs nonlinear functions, drawing on research results such as those in [26]. The observer parameters, which are used to adjust the convergence rate of the observer, are selected based on experience and fine-tuned through experimentation. For reinforcement learning, the parameters are chosen to satisfy the stability conditions derived in this paper, assuming that the system inputs and states have upper bounds. The initial values of the RLS parameters are selected from the offline controller to enhance real-time performance and convergence.

1) Simulation of offline reinforcement learning nominal controller

In the cost function, the weight coefficients are set as $\mathbf{Q}_1 = 50\mathbf{I}_6$, $\mathbf{R}_1 = 0.1\mathbf{I}_4$. The neural network structure similar to that in the ref [28]. is adopted. The output of the activation function of the critic network is 21-dimensional, and the specific form of the activation function is given as

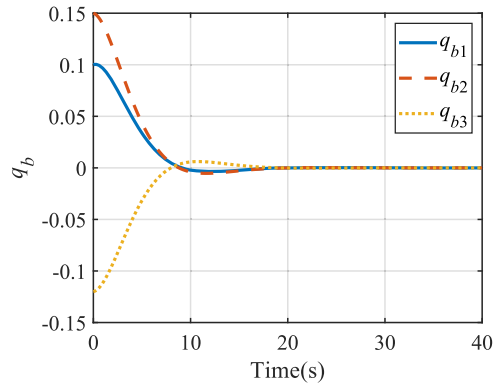


Fig. 4. Quaternions obtained by the initial control policy.

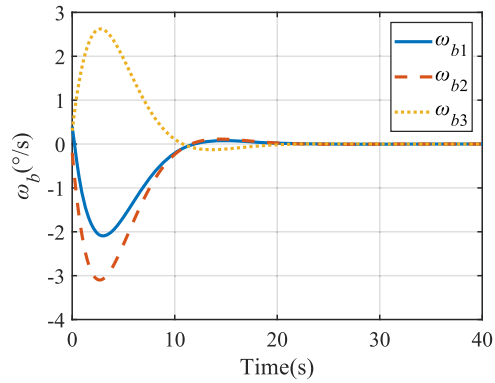


Fig. 5. Attitude angular velocity obtained by the initial control policy.

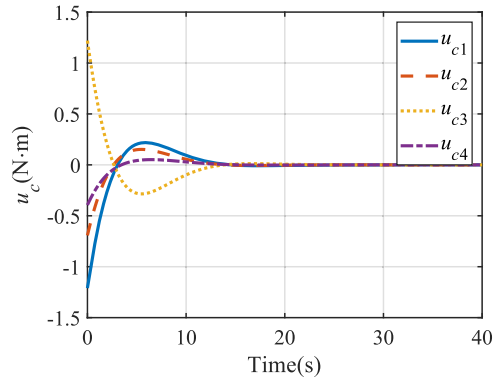


Fig. 6. Command control torque obtained by the initial control policy.

$$\sigma_{c1}(\mathbf{x}) = [x_1^2, x_2^2, x_3^2, x_4^2, x_5^2, x_6^2, x_1x_2, x_1x_3, x_1x_4, x_1x_5, x_1x_6, x_2x_3, x_2x_4, x_2x_5, x_2x_6, x_3x_4, x_3x_5, x_3x_6, x_4x_5, x_4x_6, x_5x_6]^T$$

Since the algorithm needs to start iterating from an initial admissible policy, the initial values of the critic network parameters was obtained by fitting through a PID control law with

$$\widehat{\mathbf{w}}_{c1}^0 = [0, 0, 0, 75.4, 18.9, 37.0, 0, 0, 85.3, -8.5, -11.9, 0, -8.5, 21.3, -6.0, -11.9, -6.0, 41.8, -15.1, -21.1, -10.6]^T$$

Using the initial control policy to control the nominal system, the simulation results are shown in Fig. 4-Fig. 6.

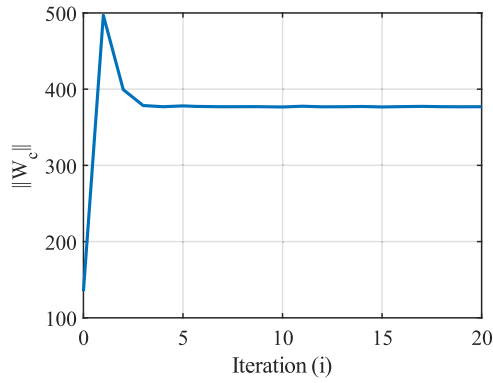


Fig. 7. The norm of critic network parameter vector obtained by offline RL algorithm.

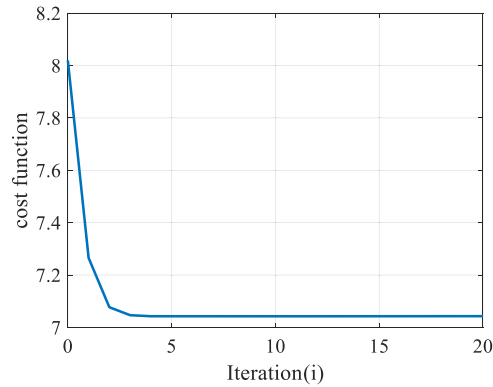


Fig. 8. The cost function obtained by offline RL algorithm.

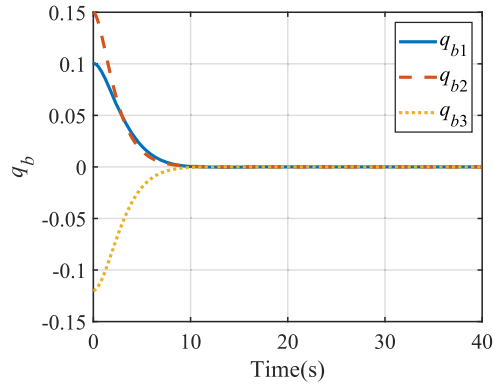


Fig. 9. Quaternions obtained by the final control policy.

From Fig. 4–Fig. 5, it can be seen the spacecraft attitude and attitude angular velocity converge eventually, but there is some overshoot and oscillation in the convergence process. Fig. 6 shows the command control torque over time, which is also convergent.

Based on the initial control policy, the nominal controller based on reinforcement learning is trained offline. The data required for iteration is obtained by state trajectories generated by 500 random initial states. Fig. 7 shows the norm of the evaluation network parameter vector under different iteration times, which converges after 4 or 5 iterations, indicating that the offline reinforcement learning algorithm has good convergence properties. Fig. 8 shows the cost function of the offline algorithm. It can be seen that the cost function continuously decreases as the number of iterations increases, and it essentially converges when the number of iterations reaches 4 to 5.

The critic network parameter vector for the final iteration is given as

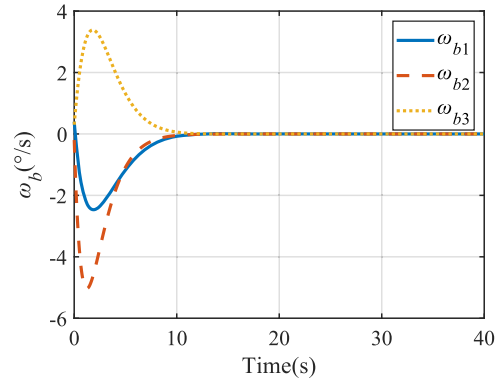


Fig. 10. Attitude angular velocity obtained by the final control policy.

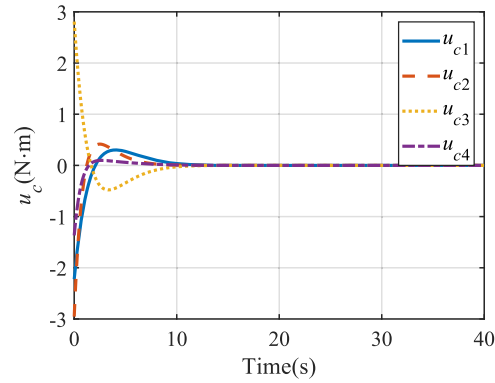


Fig. 11. Command control torque obtained by the final control policy.

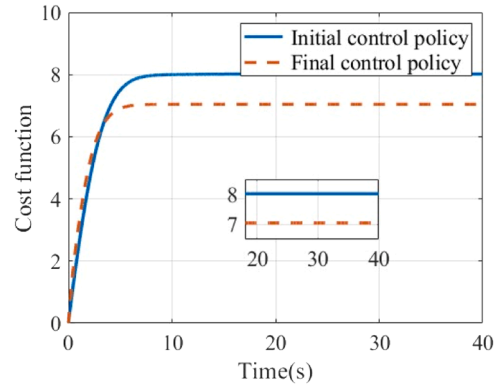


Fig. 12. Cost function obtained by the initial and final control policy.

$$\widehat{\mathbf{w}}_{c3} = [165.2, 137.7, 149.6, 128.2, 51.8, 80.2, -8.9, -9.1, 161.8, -10.5, -14.3, -7.2, -11.8, 80.4, -10.0, -13.8, -11.7, 112.6, -20.4, -27.9, -17.9]^T$$

and the simulation results using the parameters from the final iteration are shown in Fig. 9-Fig. 11.

By comparing Figs. 4 and 5 with Figs. 9 and 10, it is evident that the spacecraft attitude control performance is significantly improved after applying the reinforcement learning algorithm. The attitude convergence time is reduced, and there is almost no overshoot or oscillation during the convergence process. Fig. 11 illustrates the time-varying curve of the commanded control torque, indicating that the final control policy requires a larger control torque compared to the initial control policy.

In order to illustrate the effect of reinforcement learning algorithm more intuitively, the cost function curve is shown in Fig. 12. As can be seen from the simulation results, under the initial control policy, the cost function eventually stabilizes around 8. In contrast,

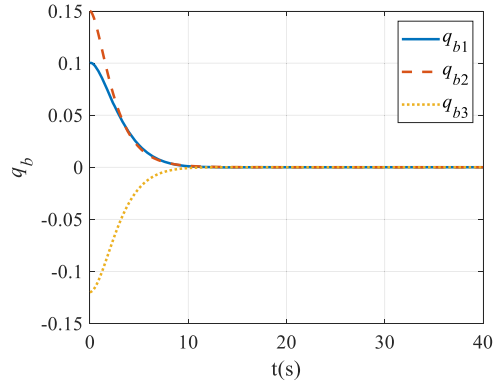


Fig. 13. Quaternions obtained by the co-state approximation-based final control policy.

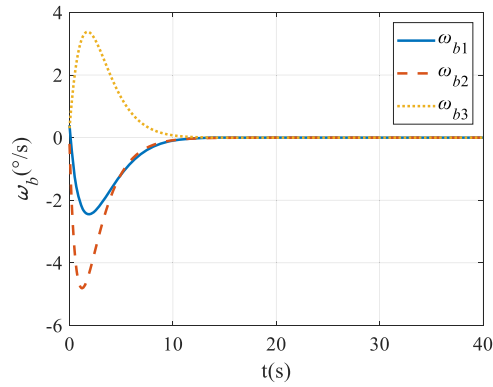


Fig. 14. Attitude angular velocity obtained by the co-state approximation-based final control policy.

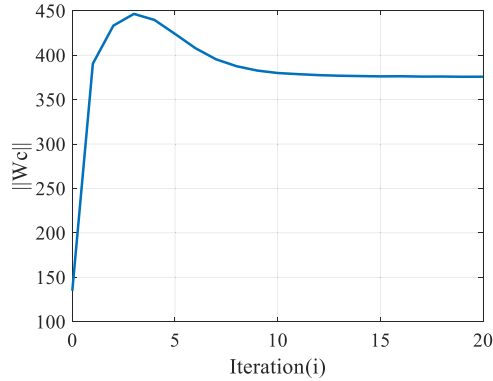


Fig. 15. The norm of critic network parameter vector obtained by offline co-state approximation-based RL algorithm.

after incorporating the offline reinforcement learning algorithm, the cost function corresponding to the final control policy decreases to around 7. This represents a reduction of approximately 12.5 %, indicating that the reinforcement learning algorithm is effective.

In Section 3.1.2, we mentioned that in addition to using neural networks to approximate the optimal value function J_1^* , it is also possible to approximate the co-state ∇J_1^* using neural networks and then obtain the approximate optimal controller through policy iteration. To compare these two methods, the numerical simulation results based on neural network approximation of the co-state are presented in Fig. 13-Fig. 16. Figs. 13 and 14 present the curves of quaternions and angular velocity over time under the final control policy obtained by using neural networks to approximate the co-state and through policy iteration. Comparison reveals that the final control performance of the two approximation methods is essentially the same. Figs. 15 and 16 illustrate the convergence of the norm of the neural network parameter vector and the cost function during the policy iteration process when using neural networks to approximate the co-state. It is observed that the convergence is relatively slow, requiring approximately 10 iterations to stabilize.

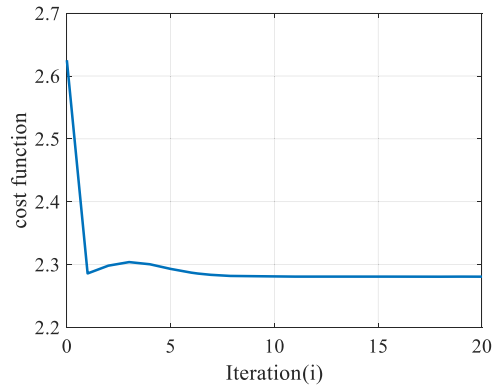


Fig. 16. The cost function obtained by offline co-state approximation-based RL algorithm.

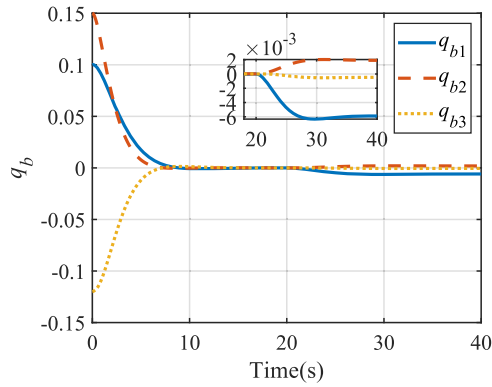


Fig. 17. Quaternions obtained by online algorithm without feedforward compensation.

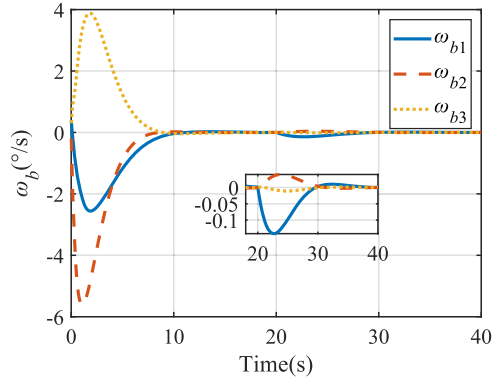


Fig. 18. Attitude angular velocity obtained by online algorithm without feedforward compensation.

2) Simulation of online fault-tolerant controller based on reinforcement learning

The initial parameters of neural network-based observer are given as follows. $\hat{\mathbf{x}}_0$ is equal to the initial states, $\hat{\mathbf{B}}_0$ is equal to the input matrix of nominal system, $\hat{\delta}_0$ is zero, $\hat{\mathbf{W}}_0$ is randomly selected in $[-0.5, 0.5]$. The activation function in observer is $\sigma(\mathbf{x}) = [\mathbf{x} \quad \sigma_{c2}(\mathbf{x})]^T$, the gain matrix is given as $\mathbf{K}_o = 20\mathbf{I}_6$ and various parameters in update law are $\alpha_w = 10$, $\alpha_b = 10$, $\alpha_\delta = 100$, $k_w = k_b = k_\delta = 0$.

The weight coefficients in the cost function and the activation function of the critic network in online reinforcement learning algorithm are kept consistent with the offline algorithm, which are $\mathbf{Q}_2 = 50\mathbf{I}_6$, $\mathbf{R}_2 = 0.1\mathbf{I}_4$, $\sigma_{c2}(\mathbf{x}) = \sigma_{c1}(\mathbf{x})$. The initial values of critic network parameters are consistent with the parameters when offline algorithm converges, and the initial covariance matrix is $\mathbf{P}(0) = 10^6\mathbf{I}_{21}$.

Before verifying the effectiveness of the online fault-tolerant controller based on reinforcement learning, the necessity of the

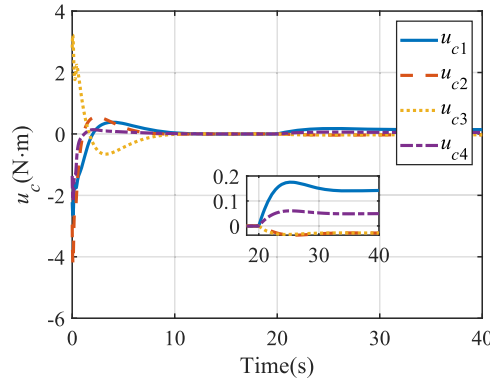


Fig. 19. Command control torque obtained by online algorithm without feedforward compensation.

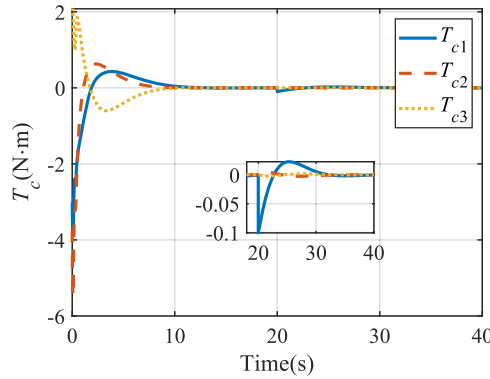


Fig. 20. Actual control torque obtained by online algorithm without feedforward compensation.

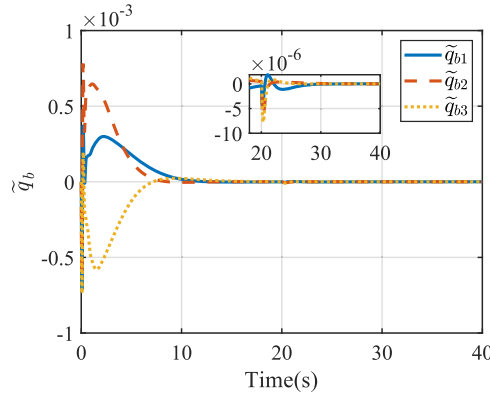


Fig. 21. Estimation errors of quaternions.

feedforward compensation algorithm is verified. The output of the feedforward compensation algorithm as $\hat{u}_s$ is set to zero, and only the observer and the online reinforcement learning algorithm are used to generate the command control torque. The simulation results are shown in Fig. 17-Fig. 20.

From Fig. 17 and Fig. 18, it can be seen that after the actuator faults occur at the 20th second, the quaternions eventually converge to non-zero values, which indicates that there is a steady-state control error if the feedforward compensation is not applied. From Fig. 19 and Fig. 20, it can be seen that when the actuator faults occur at the 20th second, there is a sudden change in the actual control torque, as well as a sudden change in quaternions and angular velocity, resulting in a change in the command control torque.

When applying the feedforward compensation algorithm, the simulation of the online fault-tolerant control scheme based on reinforcement learning is carried out, and the simulation results are shown in Fig. 21-Fig. 26.

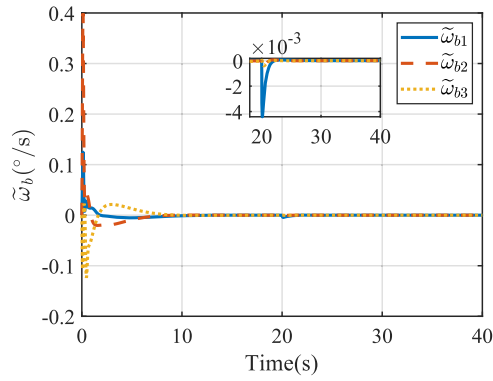


Fig. 22. Estimation errors of attitude angular velocity.

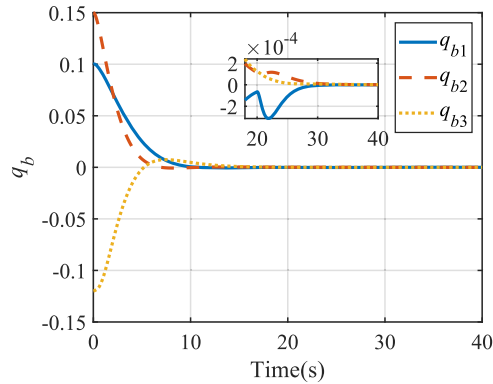


Fig. 23. Quaternions obtained by online algorithm.

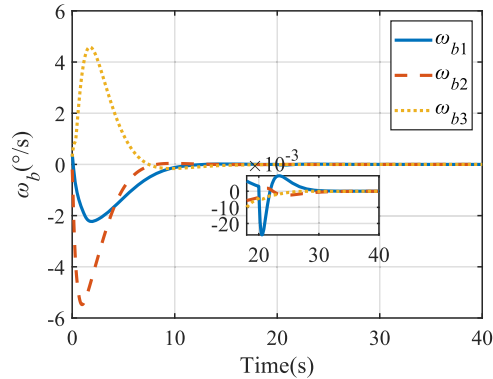


Fig. 24. Attitude angular velocity obtained by online algorithm.

From Fig. 21 and Fig. 22, it can be seen that the estimation errors of attitude quaternions and attitude angular velocity converge to zero, which indicates that the observe-based neural network can estimate the input matrix $\mathbf{B}$ and partial disturbances δ exactly.

From Fig. 23 and Fig. 24, it can be seen that due to the use of the feedforward compensation algorithm, the spacecraft attitude and attitude angular velocity can still converge after the faults occur, which indicates that the online fault-tolerant control based on reinforcement learning can effectively achieve attitude stabilization of spacecraft with inertia uncertainty, actuator faults and external disturbances.

Fig. 25 illustrates that the commanded control torque experiences a continuous deviation after 20 s to compensate for actuator faults. Due to this torque compensation, the attitude deviation and attitude angular velocity deviation that occur at the 20th second converge rapidly. Correspondingly, the actual control torque oscillates initially and then quickly converges, as shown in Fig. 26.

Fig. 27 illustrates the norm of the critic network parameter vector throughout the entire control process. It can be observed that the

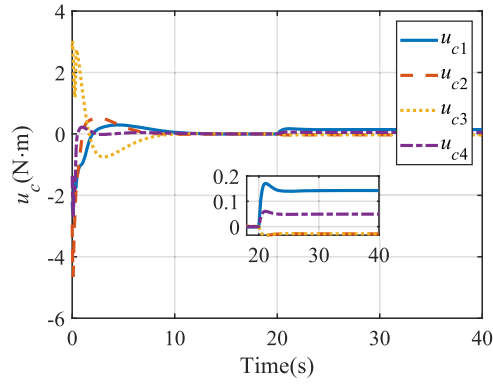


Fig. 25. Command control torque obtained by online algorithm.

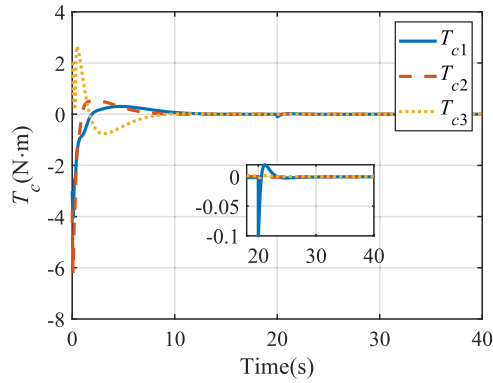


Fig. 26. Actual control torque obtained by online algorithm.

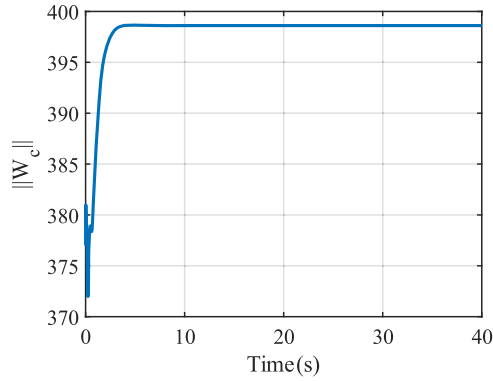


Fig. 27. The norm of critic network parameter vector obtained by online algorithm.

norm of the critic network parameter vector essentially converges before the attitude stabilization is achieved, indicating that the online reinforcement learning algorithm exhibits good convergence properties.

5. Conclusion

In this paper, a reinforcement learning-based fault-tolerant control method under the condition of unknown system model is proposed, aiming to address the attitude stabilization control problem of spacecraft in the presence of inertia uncertainty, actuator faults and external disturbances. First, an offline nominal controller based on reinforcement learning is designed. This controller serves as an approximation of the nonlinear optimal controller, thereby enhancing the intelligence and autonomy of the control system. Subsequently, an observer based on neural network is designed to estimate the system input matrix, addressing the dependency of the

offline nominal controller on the system model. The offline nominal controller is then employed as the initial control strategy, and the parameters of the critic network are updated online using the RLS method, resulting in an online reinforcement learning controller that improves the real-time performance of the algorithm. Finally, to eliminate the adverse effects of actuator additive faults and external disturbance torques on attitude control performance, a feedforward compensation algorithm based on the partial disturbances estimated by the neural network observer is designed, thereby achieving a complete online fault-tolerant control scheme based on reinforcement learning. Numerical simulation results demonstrate the effectiveness of the proposed fault-tolerant control scheme, providing a feasible solution for spacecraft attitude control.

CRedit authorship contribution statement

Shaolong Yang: Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Lei Jin:** Writing – review & editing, Validation, Supervision, Project administration, Methodology, Funding acquisition, Conceptualization. **Jiaxuan Rao:** Writing – review & editing, Writing – original draft, Software, Resources, Investigation, Formal analysis, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] M. Tafazoli, A study of on-orbit spacecraft failures[J], *Acta Astronaut.* 64 (2-3) (2009) 195–205.
- [2] A. Niederlinski, A heuristic approach to the design of linear multivariable interacting control systems[J], *Automatica* 7 (6) (1971) 691–701.
- [3] A. Levis, Challenges to control: a collective view—Report of the workshop held at the University of Santa Clara on September 18–19, 1986[J], *IEEE Trans. Automat. Contr.* 32 (4) (1987) 275–285.
- [4] J.S. Eterno, J.L. Weiss, D.P. Looze, et al., Design issues for fault tolerant-restructurable aircraft control[A], in: 1985 24th IEEE Conference on Decision and Control[C], IEEE, Fort Lauderdale, 1985, pp. 900–905.
- [5] J. Jin, S. Ko, C.K. Ryoo, Fault tolerant control for satellites with four reaction wheels[J], *Control Engineering Practice* 16 (10) (2008) 1250–1258.
- [6] C. Zhang, B. Xiao, J. Wu, et al., On low-complexity control design to spacecraft attitude stabilization: An online-learning approach[J], *Aerosp. Sci. Technol.* 110 (2021) 106441.
- [7] J. Zhou, X. Li, R. Liu, et al., Active fault-tolerant satellite attitude control based on fault effect classification[J], *Proceedings of the Institution of Mechanical Engineers, Part G: Journal of Aerospace Engineering* 231 (10) (2017) 1917–1934.
- [8] Q. Shen, C. Yue, C.H. Goh, et al., Active fault-tolerant control system design for spacecraft attitude maneuvers with actuator saturation and faults[J], *IEEE Transactions on Industrial Electronics* 66 (5) (2018) 3763–3772.
- [9] Z. Wang, Q. Li, S. Li, Adaptive integral-type terminal sliding mode fault tolerant control for spacecraft attitude tracking[J], *IEEE Access.* 7 (2019) 35195–35207.
- [10] O. Tutsoy, D. Asadi, K. Ahmadi, et al., Robust reduced order thau observer with the adaptive fault estimator for the unmanned air vehicles[J], *IEEE Trans. Veh. Technol.* 72 (2) (2023) 1601–1610.
- [11] J. Sanwale, S. Salahuddin, D.K. Giri, Neuro-Adaptive Fault-Tolerant Sliding Mode Controller for Spacecraft Attitude Stabilization[J], *J. Spacecr. Rockets.* 58 (6) (2021) 1924–1929.
- [12] N. Zhou, Y. Kawano, M. Cao, Neural network-based adaptive control for spacecraft under actuator failures and input saturations[J], *IEEE Trans. Neural Netw. Learn. Syst.* 31 (9) (2019) 3696–3710.
- [13] R.S. Sutton, A.G. Barto, *Reinforcement learning: An introduction*[M], MIT press, 2018, pp. 1–18.
- [14] M.H. Li, H. Zhang, M.L. Liu, et al., Fault tolerant control method of manipulator based on deep reinforcement learning[J], *Transducer and Microsystem Technologies* 39 (1) (2020) 53–55.
- [15] I. Ahmed, H. Khorasgani, G. Biswas, Comparison of model predictive and reinforcement learning methods for fault tolerant control[J], *IFAC-PapersOnLine* 51 (24) (2018) 233–240.
- [16] I. Ahmed, M. Quinones-Grueiro, G. Biswas, Fault-Tolerant Control of Degrading Systems with On-Policy Reinforcement Learning[J], *IFAC-PapersOnLine* 53 (2) (2020) 13733–13738.
- [17] D. Zhang, Z. Gao, Fault tolerant control using reinforcement learning and particle swarm optimization[J], *IEEE Access.* 8 (2020) 168802–168811.
- [18] J. Ren, J.W. Liu, P. Yang, Fault-tolerant tracking control for continuous flight control system based on reinforcement learning algorithm with incremental strategy[J], *Control Theory and Technology* 37 (7) (2020) 1429–1438.
- [19] M.H. Zheng, Y.H. Wu, C.Y. Li, Reinforcement learning strategy for spacecraft attitude hyperagile tracking control with uncertainties[J], *Aerosp. Sci. Technol.* 119 (2021) 107126.
- [20] C. Hua, S.X. Ding, Y.A. W. Shardt, A New Method for Fault Tolerant Control through Q-Learning[J], *IFAC-PapersOnLine* 51 (24) (2018) 38–45.
- [21] T.S. Li, W.W. Bai, Q. Liu, et al., Distributed Fault-Tolerant Containment Control Protocols for the Discrete-Time Multiagent Systems via Reinforcement Learning Method[J], *IEEE Trans. Neural Netw. Learn. Syst.* 34 (8) (2023) 3979–3991.
- [22] L.M. Wang, X.Y. Li, F.R. Gao, et al., Reinforcement Learning-Based Optimal Fault-Tolerant Tracking Control of Industrial Processes[J], *Ind. Eng. Chem. Res.* 62 (39) (2023) 16014–16024.
- [23] X.Y. Li, Q.W. Luo, L.M. Wang, et al., Off-policy reinforcement learning-based novel model-free minmax fault-tolerant tracking control for industrial processes[J], *J. Process. Control* 115 (2022) 145–156.
- [24] H. Li, Y. Wu, M. Chen, Adaptive fault-tolerant tracking control for discrete-time multiagent systems via reinforcement learning algorithm[J], *IEEE Trans. Cybern.* 51 (3) (2020) 1163–1174.
- [25] W. Zhao, H. Liu, F.L. Lewis, Fault-Tolerant Control for the Formation of Multiple Unknown Nonlinear Quadrotors via Reinforcement Learning[J], *IFAC-PapersOnLine* 53 (2) (2020) 2465–2470.
- [26] H. Zhang, K. Zhang, Y. Cai, et al., Adaptive fuzzy fault-tolerant tracking control for partially unknown systems with actuator faults via integral reinforcement learning method[J], *IEEE Transactions on Fuzzy Systems* 27 (10) (2019) 1986–1998.
- [27] Liu X., Yuan Z., Gao Z., et al. Reinforcement Learning-Based Fault-Tolerant Control for Quadrotor UAVs Under Actuator Fault[J]. 2024, 20(12): 13926–13935.
- [28] D. Vrabie, F. Lewis, Neural network approach to continuous-time direct adaptive optimal control for partially unknown nonlinear systems[J], *Neural Networks* 22 (3) (2009) 237–246.